
Network-aware Credibility and Propagation System for Local News Verification ### A Report submitted ### in partial fulfillment of the requirement ### for the award of a Degree of ## Bachelor of Technology ## in ## Computer Science and Engineering ### by **Nitesh Kumar Jhagat** **(2021BCSE0XX)** ### Under the guidance of **Dr. [Guide Name]** **Department of Computer Science and Engineering** **National Institute of Technology, Srinagar,** **Kashmir 190006, INDIA** **June 2026**

i — Department of Computer Science and Engineering, National Institute of Technology Srinagar

ii — Department of Computer Science and Engineering, National Institute of Technology Srinagar

CERTIFICATE

This is to certify that the project report entitled **Network-aware Credibility and Propagation System for Local News Verification** submitted by **Nitesh Kumar Jhagat (2021BCSE0XX)** to the Department of Computer Science and Engineering, National Institute of Technology Srinagar, Kashmir, in partial fulfillment for the award of the degree of B.Tech in Computer Science and Engineering is a bona fide record of project work carried out by him under my supervision.

Dr. [Guide Name] Supervisor Department of Computer Science and Engineering National Institute of Technology Srinagar

iii — Department of Computer Science and Engineering, National Institute of Technology Srinagar

iv — Department of Computer Science and Engineering, National Institute of Technology Srinagar

STUDENT DECLARATION

I declare that this project report titled **Network-aware Credibility and Propagation System for Local News Verification** submitted in partial fulfillment of the degree of B.Tech in Computer Science and Engineering is a record of original work carried out by me under the supervision of Dr. [Guide Name]. The matter embodied in this project, in full or in parts, have not been submitted to any other Institution or University for the award of any degree or diploma. I also declare that the work submitted by me is entirely original, free from plagiarism, and has been diligently checked through Turnitin software to ensure its authenticity.

Nitesh Kumar Jhagat

2021BCSE0XX

Srinagar, 190006 June 2026

v — Department of Computer Science and Engineering, National Institute of Technology Srinagar

vi — *Department of Computer Science and Engineering, National Institute of Technology Srinagar*

ACKNOWLEDGEMENTS

I consider it as my privilege to express gratitude and respect to all those who guided and inspired me in the completion of my B.Tech project. First of all, I would like to acknowledge and extend my heartfelt gratitude to my guide, Dr. [Guide Name], Assistant Professor at the Department of Computer Science and Engineering, National Institute of Technology, Srinagar for his valuable guidance, constant encouragement, and kind help at various stages for the execution of this dissertation work. An erudite teacher, a magnificent person, and a strict disciplinarian, I consider myself fortunate to have worked under his supervision.

I would also like to express my sincere thanks to the Department of Computer Science and Engineering at NIT Srinagar for providing me with the opportunity to work on this important project. The infrastructure and computational resources made available by the department were instrumental in the successful execution of this work.

Special thanks go to my peers and batchmates who provided thoughtful feedback during presentations and helped me identify edge cases in system testing. Their suggestions regarding bot behavior patterns and location spoofing scenarios were especially helpful during the simulation phase.

I also place on record my sense of gratitude to one and all, who directly or indirectly, have lent their hand in this venture. Last but not the least, I am deeply grateful to my family for their unwavering support and encouragement throughout the duration of this project.

Nitesh Kumar Jhagat

vii — Department of Computer Science and Engineering, National Institute of Technology Srinagar

ABSTRACT

Most social platforms decide what we see by how much we react to it. Likes, shares, and watch-time are easy to measure and easy to game, and they say almost nothing about whether a claim is true. The gap is widest for local news. When something happens on your street — a road closure, a water cut, an accident — you need to know quickly, but there is rarely a newsroom left to confirm it. This report describes a system built around a different question: what if reach were earned by credibility rather than by attention?

The Network-aware Credibility and Propagation System (NCPS) is a web platform that ranks and spreads community reports by an estimated probability that they are true. Each user is treated as an imperfect sensor with a reliability that the system has to learn, and post credibility is computed as a Bayesian posterior over votes weighted by those reliabilities. The estimate combines six cooperating components: a user engine that maintains confidence-corrected reliability, a post engine that aggregates weighted evidence, a graph engine that propagates trust between users and flags coordinated voting, a spatial layer that scores location consistency and gates how far a report travels, an optional learned layer (logistic regression plus a TF-IDF memory of resolved posts), and a decision engine whose five-condition gate has to pass before a report is allowed to reach a wider radius. A deliberate asymmetry runs through the design: network and behavioural signals can pull a user's trust down, but never inflate it above their own earned reliability.

The system was built in stages rather than at once. The baseline (Phase 1) is pure Bayesian scoring; graph trust (Phase 3), spatial checks (Phase 4), the learned layer (Phase 5), and five extended behavioural signals (Phase 6) were added one at a time so each could be measured against the last. Under a coordinated-attack simulation — forty honest users, twenty bots in four colluding groups, five adversarial and five noisy accounts — accuracy rises from 0.80 at the baseline to 1.00, and the bots' attack-success rate falls from 0.40 to 0.00, with anomaly precision reaching 1.00. The platform itself is a FastAPI backend, a React frontend, and PostgreSQL, with a standalone simulation harness used for all of the evaluation reported here.

Keywords: misinformation detection, Bayesian credibility estimation, trust propagation, anomaly detection, local news verification, coordinated-behaviour detection, spatial trust

ix — Department of Computer Science and Engineering, National Institute of Technology Srinagar

June 2026 Product Expansion Summary

The implementation was extended from a credibility-scoring prototype into a fuller deployment-ready local news platform without changing the core NCPS scoring algorithm. The new product layer adds Google authentication, category and global feed modes, hyperlocal alert delivery, an alerts inbox, browser notification subscription storage, explainable decision traces, social actions, city leaderboards, gamification, observability metrics, and mobile-focused UI refinements.

Key additions include:

- Google sign-in through backend ID-token verification and account linking.
 - Push-notification readiness with stored Web Push subscriptions, plus real-time Server-Sent Events for in-app alerts.
 - Hyperlocal alerts using existing credibility, urgency, variance, and distance signals, including the required 1 km neighborhood behavior.
 - Explainable AI module exposing C_Bayes, final credibility, evidence mass, variance, vote contribution weights, propagation gates, alert gates, and proximity checks.
 - Social features: share tracking, bookmarks, and suspicious-content reporting.
 - City leaderboard and badge/streak gamification to encourage reliable community participation.
 - Observability dashboard and metrics endpoint for deployment health, latency, response classes, and product-event counters.
 - React UI polish with a calmer news-product visual system, stronger mobile layouts, real empty/loading/error states, and map/feed/category workflows.
-

x — Department of Computer Science and Engineering, National Institute of Technology Srinagar

TABLE OF CONTENTS

DESCRIPTION	PAGE NUMBER
CERTIFICATE	iii
DECLARATION	v
ACKNOWLEDGEMENTS	vii
ABSTRACT	ix
LIST OF FIGURES	xiii
LIST OF TABLES	xv
ABBREVIATIONS / NOTATIONS / NOMENCLATURE	xvii
1. INTRODUCTION	1
1.1 Overview	1
1.1.1 The Misinformation Crisis in Digital Media	2
1.1.2 Engagement-Driven vs. Credibility-Driven Systems	3
1.2 Motivation	5
1.3 Problem Statement	6
1.4 Objectives	7
1.5 Scope and Limitations	8
1.6 Organization of Report	9
2. LITERATURE SURVEY	11
2.1 Content-Based Fake News Detection	11
2.2 Source and User Credibility Assessment	13
2.3 Propagation-Based and Graph-Based Detection	15
2.4 Truth Discovery and Crowdsourced Verification	17
2.5 Location-Aware Information Systems	19
2.6 Trust Propagation in Networks	20
2.7 Anomaly Detection in User Behavior	21
2.8 Summary of Literature Survey	22
3. PROPOSED METHODOLOGY	25
3.1 Problem Statement	25
3.2 System Architecture Overview	26
3.3 Detailed Architecture	28
3.3.1 Event-Driven Data Model	28
3.3.2 User State Computation Pipeline	29
3.3.3 Post Credibility Computation Pipeline	31
3.3.4 Graph Trust and Coordination Detection	33
3.3.5 Spatial Trust and Location Confidence	35
3.3.6 ML and Memory Augmentation	37
3.3.7 Propagation Decision Engine	38
3.4 Mathematical Formulation	39
3.5 Input Signals	42
3.6 Algorithms Used	44

DESCRIPTION	PAGE NUMBER
3.7 System Phases	48
3.8 Technology Stack	49
4. RESULTS AND DISCUSSION	51
4.1 Evaluation Metrics	51
4.2 Simulation Configuration	53
4.3 Phase-wise Experimental Evaluation	54
4.4 Frontend Implementation Screenshots	58
4.5 Functional Testing	62
4.6 Stress Test Results	63
4.7 Discussion	64
5. CONCLUSION AND FUTURE WORKS	67
5.1 Summary of Contributions	67
5.2 Key Findings	68
5.3 Future Work	69
REFERENCES	71
APPENDIX A — API Endpoint Reference	75
APPENDIX B — Hyperparameter Reference	77
APPENDIX C — Setup and Execution Guide	79

xi — Department of Computer Science and Engineering, National Institute of Technology Srinagar

xii — Department of Computer Science and Engineering, National Institute of Technology Srinagar

LIST OF FIGURES

FIGURE	TITLE	PAGE NUMBER
1.1	Comparison of Engagement-Driven vs. Credibility-Driven News Dissemination Systems	4
1.2	General NCPS Architecture Overview	8
3.1	NCPS System Architecture (Detailed Layer View)	27
3.2	User Weight Computation Pipeline	30
3.3	Post Credibility Computation Pipeline	32
3.4	Graph Trust and Coordination Detection Workflow	34
3.5	Spatial Trust and Propagation Decision Workflow	36
3.6	Database Entity-Relationship Diagram	40
4.1	Phase-wise Performance Comparison Charts	55
4.2	Login and Authentication Interface	58
4.3	User Registration Page	59
4.4	Home Feed Dashboard	60
4.5	Interactive Map Explorer	61
4.6	Alerts Inbox	62
4.7	City Leaderboard	63
4.8	User Profile	64
4.9	Mobile Responsive Layout	65

xiv — Department of Computer Science and Engineering, National Institute of Technology Srinagar

LIST OF TABLES

TABLE	TITLE	PAGE NUMBER
2.1	Summary of Literature Survey	22
3.1	Complete List of 14 Input Signals	42
3.2	System Development Phases	48
3.3	Backend Technology Stack	49
3.4	Frontend Technology Stack	50
4.1	Simulation Configuration Parameters	53
4.2	Phase-wise Evaluation Metric Results	54
4.3	Functional Test Results	63
4.4	Stress Test Scenarios and Results	64
B.1	Complete Hyperparameter Reference	77

xv — Department of Computer Science and Engineering, National Institute of Technology Srinagar

ABBREVIATIONS / NOTATIONS / NOMENCLATURE

Abbreviations

Abbreviation	Full Form
NCPS	Network-aware Credibility and Propagation System
API	Application Programming Interface
REST	Representational State Transfer
ORM	Object Relational Mapping
JWT	JSON Web Token
SQL	Structured Query Language
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheets
ML	Machine Learning
NLP	Natural Language Processing
TF-IDF	Term Frequency — Inverse Document Frequency
GNN	Graph Neural Network
CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory
UI	User Interface
ER	Entity Relationship
PK	Primary Key
FK	Foreign Key
CRUD	Create, Read, Update, Delete
HTTP	Hyper Text Transfer Protocol
CORS	Cross-Origin Resource Sharing
D3.js	Data-Driven Documents (JavaScript Library)

Notations

Symbol	Description
α, β	Bayesian prior parameters (positive and negative evidence counts)
R_i	Raw reliability of user i
R^*_i	Effective reliability (confidence-adjusted)
$Conf_i$	Confidence factor for user i
Exp_i	Experience score for user i
$Anom_i$	Anomaly score for user i
T_i	Graph-based trust score for user i
w_i	Final weight of user i
C_{Bayes}	Bayesian credibility of a post
C_{ML}	Machine learning predicted credibility
C_{memory}	Memory-based credibility from historical similarity
C_{final}	Final combined credibility score
S^+, S^-	Positive and negative weighted evidence sums

Symbol	Description
N	Total evidence mass
Var	Weighted variance of votes
λ	Damping factor for trust propagation
A_{norm}	Row-normalized adjacency matrix
σ	Standard deviation for Gaussian distance decay

xvii — Department of Computer Science and Engineering, National Institute of Technology Srinagar

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

Over the past decade the way people get their news has changed completely. Social platforms and user-generated content have replaced the editors and newsdesks that once decided what reached the public. That shift brought real gains — citizen journalism, voices that were previously ignored, and the ability to share what is happening during an emergency within seconds. It also removed the layer of checking that used to sit between a claim and its audience, and misinformation has filled the space that was left.

The World Economic Forum lists digital misinformation among its leading global risks year after year. The reasoning is not hard to follow: during disasters and health emergencies, people act on what they read, and when reliable and fabricated reports look identical, some of those decisions cause harm. The volume alone defeats manual review — no team of fact-checkers can keep pace with what a single city posts in an hour.

The harder issue is that today's platforms were never built to sort truth from falsehood in the first place. Facebook, X, Instagram and the rest optimise for engagement, so a post's reach depends on the clicks and reactions it provokes rather than on whether it is accurate. Vosoughi et al. found that false stories travel faster and reach more people than true ones, largely because falsehood tends to be more novel and more emotionally charged [1]. Engagement-ranking therefore does not merely fail to stop misinformation; it actively rewards it.

This project starts from the opposite premise. Instead of asking how much attention a report attracts, it asks how likely the report is to be true, and lets that estimate decide how far the report should spread. That single change of objective is the idea behind the Network-aware Credibility and Propagation System (NCPS).

1 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

1.1.1 The Misinformation Crisis in Digital Media

The numbers put the scale in perspective. In the Vosoughi, Roy and Aral study of Twitter, false stories were about 70% more likely to be retweeted than true ones and reached their first 1,500 readers roughly six times faster [1]. That head start is not an accident of any one platform; it follows from how engagement ranking works in general. A handful of mutually reinforcing factors keep the problem alive.

The first is amplification. Ranking systems reward interaction, and the posts that draw the most interaction are usually the ones that provoke outrage, fear or surprise — emotions that fabricated and sensational content triggers more reliably than careful reporting does.

The second is sheer volume. Nobody can personally verify the stream of posts they scroll past in a day, and in practice readers spend only a second or two deciding whether to share each one [2]. Verification loses the race before it starts.

The third is the shape of the network itself. Personalised feeds tend to surface what a reader already agrees with, so corrections struggle to reach the people who saw the original claim. Once a false narrative settles into a community it is very hard to dislodge.

The fourth, and the one this project is most concerned with, is coordination. Organised campaigns — bot fleets and paid actors pushing the same message — exploit the openness of these platforms, and their tactics keep getting more sophisticated. A system that treats every account as an independent, equally trustworthy voter has no defence against a group that votes as one.

The fifth is specific to local news: as local journalism has shrunk, many communities have no professional source left for events in their own neighbourhood, and that vacuum gets filled by unverified posts and forwarded messages.

None of this is hypothetical. During COVID-19 the WHO described the accompanying flood of false claims as an "infodemic"; bad information about treatments and vaccines spread fast enough to change real behaviour. In India, misinformation forwarded over WhatsApp has been tied to mob-violence incidents — about as concrete a consequence as one can point to [3].

2 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

1.1.2 Engagement-Driven vs. Credibility-Driven Systems

It helps to set the two models side by side, because the whole design follows from the difference between them.

The engagement model (what platforms do today). Here a post's visibility is mostly a function of how much interaction it draws. The platform logs likes, shares, comments and dwell time, and content that scores well on those signals is pushed into more feeds. That creates a feedback loop — more views produce more engagement, which earns still more views. The trouble is that engagement and truth are not the same variable, and often they pull in opposite directions. A shocking but false claim about a local politician can rack up enormous engagement, while the sober correction that follows it barely registers. Rank by engagement and you promote the first and bury the second, which is precisely backwards for an information system.

The credibility model (what NCPS does). NCPS measures credibility instead of attention. When someone files a report — say, a road accident or a water-supply outage — the platform does not just tally interactions. It does five things. It treats each voter as a noisy sensor with an estimated reliability, so votes are not counted equally. It folds those weighted votes into a Bayesian estimate of how likely the report is to be true. It watches for anomalous behaviour — burst voting, coordinated agreement, impossible location jumps. It propagates trust across the user graph so that network structure, not just individual history, informs the score. And it gates geographic reach, expanding a report's radius only once it clears credibility thresholds.

The payoff of this arrangement is resistance to manipulation. If a block of bot accounts upvotes a false report in unison, that very synchrony is what gives them away: the graph engine reads their pairwise similarity as coordination, the anomaly signals fire, and their weights collapse — so the votes that were meant to push the report up barely move it.

The following figure illustrates the key difference between these two paradigms:

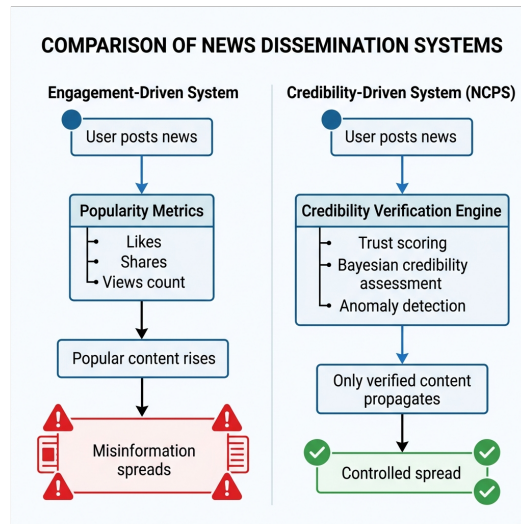


Fig 1.1: Comparison of Engagement-Driven vs. Credibility-Driven News Dissemination Systems

In Fig 1.1 the engagement pipeline on the left lets misinformation through because a popularity count cannot tell true from false. The credibility pipeline on the right inserts trust scoring, Bayesian assessment and anomaly detection between a report and its audience, so only content that survives those checks earns wider reach. That substitution — credibility where engagement used to be — is the architectural decision the rest of this report builds on.

1.2 MOTIVATION

Several things came together to make this feel like the right project at the right time.

The first is the local-news gap. Big national stories still get covered by organisations that have editors and fact-checkers; neighbourhood-scale events usually do not. Newsroom employment in the United States fell by roughly a quarter between 2008 and 2020 [4], and the picture in India is similar, with many districts effectively having no working journalists at all. So accidents, outages and safety incidents are reported first — and sometimes only — through social media and messaging groups, with no one in a position to confirm them.

The second is that the obvious fixes do not fit this setting. Manual fact-checking is thorough but slow, and for a time-sensitive local report the harm is often done before a checker even looks at it. Content-based machine learning — classifying text, images or video as fake — needs large labelled datasets that simply do not exist for local events, and it tends to stumble on the novel incidents that matter most. Platform-level tools such as community notes or fact-checking partnerships are tuned for content that has already gone viral; they do not scale down to a report that is only relevant to a few hundred people on one side of a city.

The third is the trust problem, which sits underneath the others. Most systems treat every account as equal, so a brand-new anonymous user posts with the same standing as a long-time contributor who has been right a hundred times. That even-handedness is well meant but easy to abuse. NCPS instead keeps a per-user trust profile — drawn from past reliability, position in the interaction graph, geographic consistency, and behavioural patterns — and uses it to weight how much each vote and report actually counts.

The fourth is geography, which local news cannot ignore. A crash in South Delhi matters a great deal a kilometre away and almost not at all in Mumbai, yet mainstream feeds barely encode that. NCPS makes distance a first-class input: a report's radius grows only as its credibility does, so an unconfirmed claim does not get to alarm a city it has nothing to do with.

The last is a stance rather than a feature. In any information system, wrongly amplifying a falsehood usually costs more than briefly holding back something true. The design takes that asymmetry seriously: network and behavioural signals can only lower a user's trust, never inflate it past what they have earned. Choosing to make the system cautious by construction is what makes it hard to game.

5 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

1.3 PROBLEM STATEMENT

Given a stream of user-generated local news reports and community votes on those reports, the problem is to:

1. **Estimate the credibility** of each report in near-real-time, based on the weighted opinions of community members whose individual reliabilities are themselves uncertain and must be estimated simultaneously.
2. **Detect and mitigate manipulation attempts** including coordinated bot attacks, location spoofing, and adversarial voting strategies, without requiring manual moderation.
3. **Control the geographic propagation** of reports such that only sufficiently credible content is allowed to spread beyond its immediate locality, preventing unverified claims from reaching distant communities.

Formally, the problem can be stated as follows:

Given a set of users $U = \{u_1, u_2, \dots, u_n\}$, a set of posts $P = \{p_1, p_2, \dots, p_m\}$, and a set of interactions $I = \{(u_i, p_j, v_{ij})\}$ where $v_{ij} \in \{-1, +1\}$ represents user u_i 's vote on post p_j , compute:

- **User weights** $w = \{w_1, w_2, \dots, w_n\}$ where each w_i reflects the estimated trustworthiness of user u_i , incorporating reliability, experience, anomaly detection, and graph-based trust propagation.
- **Post credibilities** $C = \{C_1, C_2, \dots, C_m\}$ where each C_j represents the estimated truthfulness of post p_j , derived from Bayesian aggregation of weighted votes.
- **Propagation decisions** $D = \{d_1, d_2, \dots, d_m\}$ where each d_j determines whether post p_j should be allowed to expand its geographic reach based on a multi-criteria assessment.

The key challenge is the circular dependency between user weights and post credibilities: to compute accurate credibility for a post, we need reliable user weights, but to assess user reliability, we need to know the true credibility of posts they voted on. The NCPS resolves this chicken-and-egg problem through iterative Bayesian estimation with conservative priors, augmented by independent signals (graph structure, spatial behavior, temporal patterns) that break the circularity.

1.4 OBJECTIVES

The aim of the project is to design, build and evaluate a credibility-driven distribution system for local news. Concretely, the objectives are:

1. **Design a Bayesian credibility estimation framework** that treats users as noisy sensors with uncertain reliabilities and computes posterior credibility scores for user-submitted reports, incorporating confidence-corrected user weights.
2. **Develop a multi-dimensional user trust model** that goes beyond simple reputation scores by incorporating:
 3. Bayesian reliability estimation with confidence correction
 4. Experience-weighted contribution (logarithmic scaling)
 5. Multi-signal anomaly detection (burst, entropy, consensus, coordination, location)
 6. Graph-based trust propagation with safety constraints
7. **Implement graph-based coordination detection** to identify clusters of users engaging in coordinated manipulation, using agreement-frequency-temporal similarity features and link dampening.
8. **Design a spatial trust subsystem** that leverages geographic information for both user verification (location consistency, device/IP patterns) and content propagation control (radius expansion governed by credibility thresholds).
9. **Integrate machine learning augmentation** to supplement Bayesian estimation with learned patterns, including a memory module that detects textual similarity with historically resolved reports.
10. **Build a conservative propagation decision engine** that uses a 5-condition AND gate to ensure only sufficiently verified content is allowed to expand its geographic reach.
11. **Develop a full-stack web application** with a React-based frontend, FastAPI backend, PostgreSQL database, and comprehensive REST API that demonstrates all system capabilities through an intuitive user interface.
12. **Create a simulation framework** for empirical validation that models diverse user populations (honest, noisy, adversarial, coordinated bots) and measures system performance across multiple metrics including accuracy, attack success rate, Brier score, and weight correlation.

1.5 SCOPE AND LIMITATIONS

Scope: NCPS is built as a working proof of concept for credibility-driven distribution of local news. It covers text reports, community voting, map-based visualisation, and the trust computation that ties them together. In its current form it provides: - Full user authentication and authorization - Report creation with geographic tagging - Community voting with credibility feedback - Interactive map-based visualization - Trust profile display and decomposition - Simulation-based validation with configurable scenarios

Limitations: The current implementation has several known limitations: - The system does not perform content analysis (NLP-based fake news detection on text/images) — it relies entirely on network-based signals - Real-world deployment would require integration with external verification sources - The simulation uses synthetic data; real-world user behavior may differ significantly from simulated patterns - The current system supports single-language (English) content only - Location services depend on browser-provided geolocation, which users can spoof at the browser level

1.6 ORGANIZATION OF REPORT

The remainder of this report is organized as follows:

Chapter 2: Literature Survey reviews prior work across the areas NCPS borrows from — content-based detection, source and user credibility, propagation- and graph-based methods, truth discovery and crowdsourced verification, location-aware systems, and trust propagation — and notes, for each, what it offers and where it falls short for the local-news case this project targets.

Chapter 3: Proposed Methodology sets out the full design: the architecture, the six computation engines, the mathematical formulation of the thirteen core formulas, the fourteen input signals, the algorithms, the phased build, and the technology stack.

Chapter 4: Results and Discussion presents the experimental evaluation of the system across all six development phases, including quantitative metrics (accuracy, attack success rate, Brier score, weight correlation, anomaly precision and recall), frontend implementation screenshots, functional test results, and stress test scenarios.

Chapter 5: Conclusion and Future Works summarizes the contributions of this project, presents key findings from the experimental evaluation, and outlines directions for future research and development.

The report concludes with references and three appendices containing the complete API endpoint reference, hyperparameter reference table, and setup/execution guide.

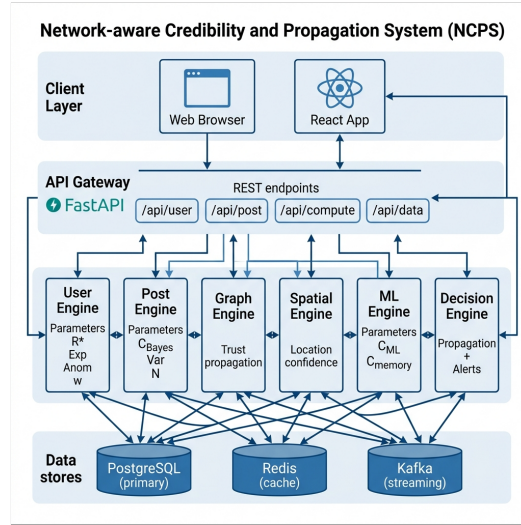


Fig 1.2: General NCPS Architecture Overview showing the layered design with Client Layer, API Gateway, six computation engines, and data stores

CHAPTER 2

LITERATURE SURVEY

The problem of misinformation detection and credibility assessment has attracted significant attention from the research community over the past decade. This chapter provides a comprehensive review of existing approaches, organized by their primary methodology. For each approach, we examine the underlying technique, its strengths, and crucially, the limitations that motivate the design choices in NCPS.

2.1 CONTENT-BASED FAKE NEWS DETECTION

Content-based approaches attempt to determine the veracity of information by analyzing the content itself — the text, images, or structural features of a news article or social media post.

1. Deep Learning for Fake News Detection (Ruchansky et al., 2017)

Ruchansky, Diaz, and Wang proposed CSI (Capture, Score, and Integrate), a hybrid deep learning model that combines an LSTM-based module for capturing temporal patterns in user engagement, a source credibility scoring module, and an integration module that produces a final fake/true classification [5]. The model was evaluated on Twitter and Weibo datasets and achieved around 95% accuracy. However, the approach requires large labeled training datasets and operates as a post-hoc classifier — it can only assess content after it has already spread, making it unsuitable for real-time credibility estimation during the early stages of information propagation.

2. Geometric Deep Learning for Social Media (Monti et al., 2019)

Monti et al. explored the use of graph convolutional networks (GCNs) for fake news detection, leveraging the geometric structure of user interaction patterns [6]. By treating the problem as a graph classification task where nodes represent users and edges represent sharing/commenting relationships, they achieved strong detection performance. The key insight is that fake and real news stories generate structurally different propagation patterns. While this approach is promising, it requires the full propagation graph to be available, which limits its applicability to early-stage detection. The NCPS, in contrast, computes credibility incrementally as each new vote arrives.

3. Multi-Modal Fake News Detection

Wang et al. (2018) proposed the EANN model (Event Adversarial Neural Networks), which uses adversarial training to learn event-invariant features for fake news detection [7]. This approach attempts to address the generalization problem — ensuring that models trained on one set of fake news stories can detect novel fabrications. Jin et al. (2017) specifically addressed multi-modal fake news by jointly analyzing text and images using attention-based RNNs [8]. While multi-modal approaches represent the state-of-the-art in content analysis, they fundamentally struggle with novel events that have no historical analogs, which is precisely the situation that characterizes breaking local news.

4. Linguistic Feature Analysis

Perez-Rosas et al. (2018) took a more traditional NLP approach, identifying linguistic cues that distinguish fake from real news, including the use of emotional language, personal pronouns, and hedging words [9]. While their analysis revealed interesting patterns (fake news tends to use more emotional and less analytic language), the feature differences are statistical tendencies rather than definitive signals. A skilled fabricator could easily adjust their writing style to avoid detection. The NCPS deliberately avoids content analysis for this reason, instead relying on network-level signals that are much harder to manipulate.

12 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

2.2 SOURCE AND USER CREDIBILITY ASSESSMENT

Rather than analyzing content directly, source-based approaches focus on evaluating the credibility of the person or entity sharing the information.

5. Source Reliability in Social Networks (Shu et al., 2019)

Shu et al. conducted a comprehensive survey of fake news detection approaches, with particular emphasis on source credibility features [2]. They identified key source-level indicators including account age, posting frequency, follower-to-following ratio, and historical accuracy. Their framework, FakeNewsNet, provided benchmark datasets

that have been widely used in subsequent research. However, source features alone are insufficient because (a) newly created accounts may be legitimate, and (b) previously reliable sources can share misinformation accidentally. The NCPS addresses this by not treating user reliability as a static label but as a continuously updated Bayesian estimate with confidence correction.

6. Credibility Propagation in Information Networks (Gupta et al., 2012)

Gupta, Zhao, and Han proposed a credibility analysis framework that models the relationships between events, sources, and tweets as a heterogeneous information network [10]. Credibility scores are propagated through the network using an iterative algorithm similar to PageRank. This approach is conceptually similar to the NCPS's graph trust propagation, but Gupta et al. do not incorporate anomaly detection or spatial signals, making their system vulnerable to coordinated manipulation attacks where a group of accounts mutually reinforce each other's credibility.

7. User Profiling for Bot Detection (Varol et al., 2017)

Varol et al. developed Botometer (formerly BotOrNot), a machine learning-based tool that evaluates over 1,000 features of a Twitter account to estimate the likelihood that it is a bot [11]. Features include temporal patterns, network structure, content analysis, and behavioral cues. While Botometer is highly effective for individual bot detection, it requires access to platform-specific APIs and extensive historical data for each account. The NCPS takes a different approach to bot detection through behavioral anomaly signals (burst patterns, coordination detection, location inconsistencies) that can identify suspicious behavior even for accounts with limited history.

13 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

8. Trust and Reputation Systems

Josang, Ismail, and Boyd (2007) provided a foundational survey of trust and reputation systems for online services [12]. They categorized approaches into centralized reputation (like eBay's feedback system), distributed reputation (where trust is computed locally by each participant), and Bayesian reputation (where trust is modeled as a probability distribution). The NCPS draws heavily from the Bayesian reputation paradigm, using Beta distributions (parameterized by α and β counts) to model user reliability. However, unlike traditional reputation systems that rely solely on direct evidence, the NCPS augments direct reliability estimates with graph-based trust propagation and multi-signal anomaly detection.

2.3 PROPAGATION-BASED AND GRAPH-BASED DETECTION

These approaches analyze how information spreads through networks, recognizing that the propagation patterns of true and false information tend to differ systematically.

9. Rumor Detection with Graph Neural Networks (Bian et al., 2020)

Bian et al. proposed Bi-GCN (Bi-directional Graph Convolutional Network), which models both the top-down propagation structure and the bottom-up aggregation structure of rumor threads [4]. By applying graph convolutions in both directions, the model captures how a rumor spreads from the source to followers (top-down) and how community responses aggregate to form a collective assessment (bottom-up). Bi-GCN achieved state-of-the-art results on Twitter15 and Twitter16 datasets. The key limitation, similar to other GNN approaches, is the requirement for a substantially developed propagation tree before classification can be performed, which introduces detection latency.

10. Cascade-Based Early Detection (Ma et al., 2018)

Ma et al. explored the use of recurrent neural networks (specifically Tree-LSTMs) to model the temporal evolution of information cascades for early rumor detection [13]. Their approach can operate with partial cascade information, making it more suitable for early detection than full-graph methods. However, it still requires a minimum cascade depth of 3-5 levels to achieve reasonable accuracy, and its performance degrades sharply for cascades with fewer than 10 nodes. In the local-news setting, where many reports attract only a handful of interactions, this limitation is particularly problematic.

11. Network-Based Truth Discovery (Li et al., 2016)

Li et al. addressed the truth discovery problem — estimating the veracity of claims when multiple sources provide conflicting information [14]. Their approach jointly estimates source reliability and claim truthfulness through an iterative optimization process. This is conceptually closely related to the NCPS's approach, where user reliability and post credibility are simultaneously estimated. However, Li et al. do not incorporate graph structure, spatial information, or anomaly detection, and their framework is designed for static datasets rather than streaming, real-time scenarios.

12. Social Network Analysis for Influence Detection (Kempe et al., 2003)

While not directly focused on misinformation, Kempe, Kleinberg, and Tardos' foundational work on influence maximization in social networks [15] established the theoretical framework for understanding how information propagates through graph structures. Their greedy algorithm for influence maximization demonstrated that network topology plays a crucial role in determining which nodes have the greatest impact on information spread. The NCPS leverages similar network-topological insights, but inverts the objective: rather than maximizing influence, we aim to minimize the spread of unreliable content by dampening the influence of untrustworthy nodes.

2.4 TRUTH DISCOVERY AND CROWDSOURCED VERIFICATION

Truth discovery approaches treat users as sources with varying reliability and attempt to estimate both source quality and claim veracity simultaneously.

13. Truth Discovery with Conflicting Information Providers (Yin et al., 2008)

Yin, Han, and Yu proposed one of the earliest truth discovery frameworks [16], addressing the fundamental challenge of determining factual truth when multiple information providers give conflicting statements. Their approach iteratively refines source reliability estimates and fact confidence scores until convergence. The key insight is that reliable sources tend to provide accurate information, and accurate information tends to be provided by reliable sources. This circular dependency is resolved through an iterative algorithm analogous to expectation-maximization. The NCPS builds upon this foundational concept but significantly extends it by incorporating temporal decay, anomaly detection, and graph-based trust signals that are not present in the original truth discovery framework.

14. Crowdsourced Fact-Checking (Allen et al., 2021)

Allen et al. (2021) studied the effectiveness of crowdsourced fact-checking, where ordinary users rate the accuracy of news articles [17]. Surprisingly, they found that aggregated crowd assessments showed high agreement with professional fact-checkers, suggesting that the "wisdom of crowds" can be leveraged for credibility assessment. However, their study also identified vulnerability to manipulation — when a sufficiently large group of users coordinates to rate false content as true, the crowd's assessment can be corrupted. This vulnerability motivated the NCPS's multi-layered defense mechanism, particularly the coordination detection module that identifies and penalizes groups of users who exhibit suspiciously similar voting patterns.

15. Community Notes (Twitter/X)

Twitter's Community Notes program (formerly Birdwatch) represents perhaps the most prominent real-world deployment of crowdsourced fact-checking [18]. The system allows users to write notes providing context or corrections to potentially misleading tweets. Notes are shown to a broad audience only if users from diverse political perspectives agree on their helpfulness. While Community Notes has shown promise for high-visibility viral content, it is fundamentally designed for a different scale — it targets tweets that have already gone viral, whereas NCPS operates on local reports that may have only a handful of interactions. Additionally, Community Notes relies on a bridging-based consensus algorithm that requires substantial participation volumes to function effectively, which is impractical for hyper-local content.

2.5 LOCATION-AWARE INFORMATION SYSTEMS

The spatial dimension of information credibility has received comparatively less attention in the literature, despite its relevance to local news verification.

16. Location-Based Social Network Analysis (Cho et al., 2011)

Cho, Myers, and Leskovec studied mobility patterns in location-based social networks, finding that social network ties significantly influence individual mobility patterns [19]. Their work demonstrated that geographic information can provide valuable signals for understanding user behavior and detecting anomalies. Users who claim to be in vastly different locations within short time spans, or who exhibit teleporting behavior, are likely either spoofing their location or using compromised accounts. The NCPS's spatial engine incorporates similar insights, using location consistency metrics and device fingerprinting to assess user trustworthiness.

17. Geospatial Information Credibility (Goodchild and Glennon, 2010)

Goodchild and Glennon explored the concept of spatial data quality in the context of volunteered geographic information (VGI), arguing that the credibility of user-contributed spatial data can be partially assessed through geographic consistency checks [20]. Reports that are geographically inconsistent (e.g., a user claiming to witness an event while located far from the reported event location) should receive lower credibility. The NCPS implements this principle through its proximity weighting function, which applies Gaussian distance decay to reduce the influence of votes from users located far from the reported event.

2.6 TRUST PROPAGATION IN NETWORKS

Trust propagation approaches leverage the transitive nature of trust — if A trusts B and B trusts C, then A has some basis for trusting C, albeit with decreased confidence.

18. EigenTrust (Kamvar et al., 2003)

Kamvar, Schlosser, and Garcia-Molina proposed EigenTrust, a reputation management algorithm for peer-to-peer networks [21]. EigenTrust computes global trust values by iteratively aggregating local trust relationships, similar to how PageRank aggregates link authority. The algorithm converges to a unique steady-state vector that represents each node's global reputation. The NCPS's graph trust propagation is inspired by EigenTrust but incorporates a critical safety constraint: trust can never be inflated beyond a user's individual reliability ($T_i \leq R_i^*$). This constraint, which is absent in EigenTrust, prevents colluding groups from artificially boosting each other's trust scores.

19. TrustRank (Gyöngyi et al., 2004)

Gyöngyi, Garcia-Molina, and Pedersen developed TrustRank for combating web spam, demonstrating how trust can be propagated from a small set of manually verified seed nodes to the rest of a web graph [22]. The key insight is that trustworthy pages tend to link to other trustworthy pages, allowing trust to be estimated for pages that have not been manually reviewed. While the NCPS does not assume the existence of pre-verified seed users, it leverages a similar propagation mechanism where well-behaved users (identified through Bayesian reliability estimation) serve as implicit trust anchors.

20 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

2.7 ANOMALY DETECTION IN USER BEHAVIOR

Detecting anomalous behavior is crucial for identifying manipulation attempts in online platforms.

20. Bot Detection Through Temporal Analysis (Cresci et al., 2017)

Cresci et al. demonstrated that simple feature-based bot detection can be evaded by sophisticated social bots, and proposed DNA-inspired techniques that model user behavior as sequences of actions [23]. By analyzing the temporal patterns of these action sequences, they could identify bots even when individual features appeared human-like. The NCPS's temporal anomaly detection (burst detection, entropy analysis) is motivated by similar observations — coordinated bots tend to exhibit temporally regular patterns that differ from the naturally irregular patterns of human behavior.

21. Coordinated Inauthentic Behavior Detection (Nizzoli et al., 2021)

Nizzoli et al. studied coordinated inauthentic behavior on social media, where groups of accounts work together to manipulate public discourse [24]. They identified key signatures of coordination, including temporal synchronization (accounts posting or engaging at the same times), content alignment (accounts sharing the same or similar content), and network clustering (accounts forming densely connected subgroups). The NCPS implements coordination detection by building a user graph based on voting agreement, temporal proximity, and interaction frequency, then applying link dampening to edges within identified coordinated clusters.

21 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

2.8 SUMMARY OF LITERATURE SURVEY

The following table provides a consolidated summary of the literature reviewed in this chapter:

Table 2.1: Summary of Literature Survey

S.No	Technique & Reference	About Technique	Application	Results	Limitations
1	CSI - LSTM + Source Scoring [5]	Hybrid deep learning combining temporal user patterns with source credibility	Fake news detection on Twitter, Weibo	~95% accuracy	Requires large labeled datasets; post-hoc classification only
2	GCN-based Detection [6]	Graph convolutional networks on propagation graphs	Social media fake news	Strong performance on Twitter datasets	Requires full propagation graph; limited early detection
3	EANN [7]	Adversarial training for event-invariant features	Multi-event fake news detection	Improved cross-event generalization	Struggles with completely novel event types
4	Linguistic Analysis [9]	Statistical NLP features (emotion, pronouns, hedging)	News article credibility	Identified useful linguistic patterns	Features are statistical tendencies; can be gamed
5	FakeNewsNet [2]	Comprehensive source credibility framework	Benchmark for fake news research	Provided widely-used datasets	Static credibility labels; no temporal dynamics
6	Credibility Propagation [10]	Heterogeneous network-based credibility flow	Event-source-tweet credibility	Effective credibility estimation	No anomaly detection; vulnerable to coordinated attacks
7	Botometer [11]	ML-based bot detection using 1000+ features	Twitter bot identification	High accuracy bot detection	Requires platform APIs; computationally expensive
8	Bayesian Reputation [12]	Beta distribution-based trust modeling	Online service reputation	Principled uncertainty handling	No graph propagation; single-signal assessment
9	Bi-GCN [4]	Bi-directional graph convolution on rumor trees	Rumor detection on Twitter	State-of-the-art on Twitter15/16	Requires developed propagation tree (latency)
10	Tree-LSTM [13]	Recursive neural networks on cascade structure	Early rumor detection	Better early detection than GCN	Needs minimum cascade depth; degrades with few nodes
11	Network Truth Discovery [14]	Joint source-claim estimation through optimization	Multi-source fact verification	Accurate truth estimation	No real-time processing; static datasets only
12	Influence Maximization [15]	Greedy algorithm for optimal influence seeds	Social network influence	Submodular optimization guarantees	Maximizes influence; doesn't address credibility
13	Truth Discovery [16]	Iterative source reliability and fact estimation	Conflicting information resolution	Effective truth discovery	No temporal decay, anomaly detection, or graph signals
14	Crowdsourced Fact-Checking [17]	Aggregated crowd assessments vs. expert judgments	News article accuracy rating	High agreement with fact-checkers	Vulnerable to coordinated manipulation
15	Community Notes [18]	Bridging-based crowdsourced annotations	Twitter context notes	Effective for viral content	Requires high participation; not for local content
16	Location-Based SNA [19]	Mobility pattern analysis in geotagged networks	User behavior understanding	Validated geographic consistency signals	Not directly applied to credibility assessment
17	Spatial Data Quality [20]	Geographic consistency for volunteered info	VGI quality assessment	Framework for spatial credibility	Conceptual; no system implementation
18	EigenTrust [21]	Iterative trust aggregation in P2P networks	Peer-to-peer reputation	Convergent global trust scores	No safety constraint; trust can be inflated by collusion
19	TrustRank [22]	Trust propagation from verified seeds	Web spam detection	Effective spam identification	Requires manually verified seed nodes
20	DNA-Based Bot Detection [23]	Temporal behavior sequence analysis	Social bot detection	Detected evasive bots	Requires long behavioral history
21	Coordinated Behavior Detection [24]	Temporal, content, and network coordination signals	Coordinated inauthentic behavior	Identified coordinated campaigns	Detection only; no integrated mitigation

Reading these strands together, five gaps stand out — and they are what NCPS is designed to close.

Most obviously, the pieces exist but nobody has put them in one box. Bayesian credibility, graph trust, anomaly detection, spatial reasoning and a learned layer each appear separately in the literature; a single framework that runs all of them together is missing. Second, almost everything is offline. The strong methods classify a story after the cascade has formed, whereas a local report needs a credibility estimate that updates with each vote as it arrives. Third, the local-news case is essentially unaddressed — the benchmarks and the algorithms assume viral content on a large

platform, not a report seen by two hundred people on one side of a city. Fourth, several systems detect coordination but stop there; detecting a bot ring is only useful if the same pass also discounts its influence, and that mitigation step is usually left out. Fifth, the idea of letting credibility govern geographic reach does not appear at all.

NCPS takes the established machinery — Beta-distribution reliability, EigenTrust-style propagation, behavioural anomaly signals — and adds what was missing for this setting: a strictly conservative trust rule, a five-condition propagation gate, and spatial trust as a first-class input, all running incrementally in one system.

24 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

CHAPTER 3

PROPOSED METHODOLOGY

This chapter lays out how NCPS actually works — the architecture, the mathematics behind each score, the algorithms, and the implementation choices that turn the idea from Chapter 1 into a running platform.

3.1 PROBLEM STATEMENT

Given the gaps identified in the literature review, we formalize the problem that NCPS aims to solve.

Input: A stream of events comprising: - User registrations with geographic coordinates - User-submitted news reports (posts) with content and location - Community votes (upvote +1 or downvote -1) on existing posts

Output: For each post, a credibility score $C \in [0, 1]$ and a propagation decision (expand/hold). For each user, a composite weight $w \in [0, 1]$ reflecting their estimated trustworthiness.

Constraints: 1. The system must be conservative — trust can decrease through manipulation but can never be artificially inflated above individual merit 2. Credibility computation must be incremental — each new vote should immediately update the post's credibility without requiring full recomputation 3. The system must be resistant to coordinated manipulation — groups of colluding accounts should not be able to game credibility scores 4. Geographic propagation must be controlled — unverified content should not spread beyond its locality

25 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

3.2 SYSTEM ARCHITECTURE OVERVIEW

The NCPS follows a layered architecture pattern with clear separation of concerns between the presentation layer, application logic, computation engines, and data persistence. The architecture is designed to be modular — each computation engine operates independently and can be added or removed without affecting the others, which enabled the phased development approach described in Section 3.7.

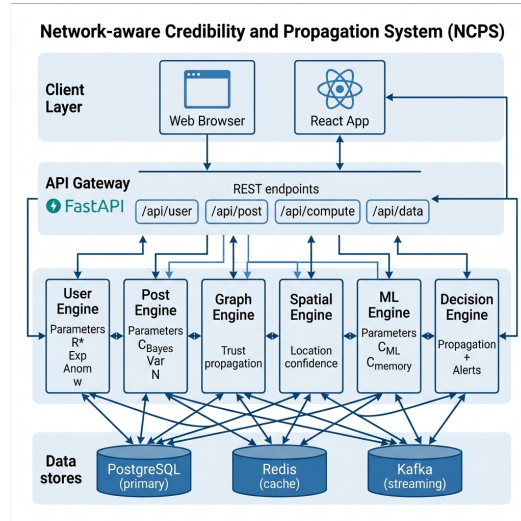


Fig 3.1: NCPS System Architecture showing four distinct layers — Client Layer (React App, Web Browser), API Gateway (FastAPI with REST endpoints), Computation Layer (six engines), and Data Layer (PostgreSQL, Redis, Kafka)

As illustrated in Fig 3.1, the system comprises four distinct layers:

Client Layer: The user-facing interface built with React (for the rich interactive dashboard) and vanilla JavaScript with HTML/CSS (for the primary web application). The client communicates with the backend exclusively through REST API calls, ensuring a clean separation between frontend and backend concerns.

API Gateway: A FastAPI-based REST API server that handles authentication (JWT-based), request validation, and routing. The gateway exposes endpoints for user management, post operations, voting, feed retrieval, map data, profile information, and analytics.

Computation Layer: Six specialized engines that collectively implement the credibility assessment pipeline: - **User Engine** (`user_engine.py`): Computes reliability, confidence, experience, and final user weights - **Post Engine** (`post_engine.py`): Computes Bayesian credibility, variance, and evidence mass - **Graph Engine** (`graph_engine.py`): Builds the user interaction graph, detects coordination, and propagates trust - **Spatial Engine** (`spatial.py`): Computes location confidence, proximity weights, and manages propagation radius - **ML Engine** (`ml_engine.py`): Provides machine learning predictions and memory-based similarity scores - **Decision Engine** (`decision.py`): Evaluates the 5-condition gate for propagation decisions and generates alerts

Data Layer: PostgreSQL serves as the primary relational database, with tables for user accounts, user state, posts, interactions, locations, and the user graph. Redis provides caching for frequently accessed data, and Kafka handles event streaming for asynchronous processing.

27 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

3.3 DETAILED ARCHITECTURE

3.3.1 Event-Driven Data Model

The NCPS processes three fundamental event types, each triggering a cascade of state updates across the computation engines:

Registration Event: When a new user registers, the system creates an authentication account and initializes a user profile with default Bayesian priors ($\alpha_0 = 1$, $\beta_0 = 1$, representing maximum uncertainty), zero experience, zero anomaly score, and the provided geographic coordinates. The initial reliability $R = \alpha/(\alpha+\beta) = 0.5$ and confidence is near zero, reflecting the principle that new users should have minimal influence until they establish a track record.

Post Event: When a user creates a post, the system stores the content, location (latitude, longitude), urgency classification, and initializes the post's credibility with agnostic priors ($\alpha_0 = 1$, $\beta_0 = 1$, giving $C_Bayes = 0.5$). The post is assigned an initial propagation radius based on the user's weight and the urgency level.

Vote Event: This is the most complex event type and triggers the majority of computation. When user i votes on post j : 1. The user's current weight w_i is fetched (or computed if stale) 2. The vote ($v \in \{-1, +1\}$) is weighted by w_i and applied to the post's evidence accumulators 3. The post's Bayesian credibility is recomputed 4. If anomaly detection is enabled, the user's behavioral signals are updated 5. If graph trust is enabled, the user graph edges are updated based on vote agreement patterns 6. The propagation decision is re-evaluated

This event-driven model ensures that credibility scores are always up-to-date and that the system responds immediately to new information. There is no batch processing step — every vote triggers an immediate incremental update.

28 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

3.3.2 User State Computation Pipeline

The User Engine maintains a comprehensive state profile for each user, combining multiple signals into a single composite weight. The pipeline, illustrated in Fig 3.2, proceeds through several stages.

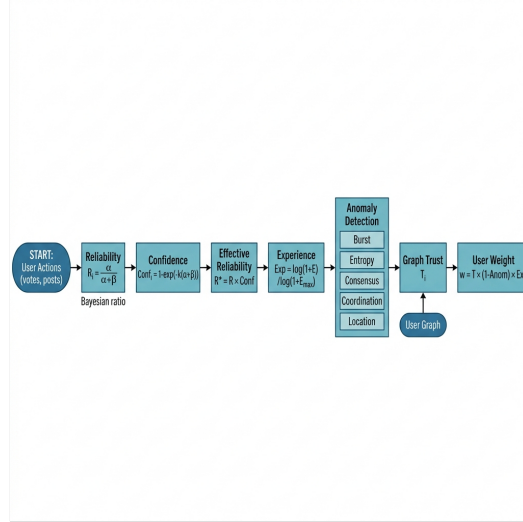


Fig 3.2: User Weight Computation Pipeline showing the flow from raw user actions through Bayesian reliability, confidence correction, experience weighting, anomaly detection, graph trust, to the final composite weight

Stage 1: Raw Reliability Estimation

Each user is modeled as a Bernoulli information source with unknown reliability parameter p_i . We maintain conjugate Beta prior counts α_i (positive evidence) and β_i (negative evidence). The raw reliability is simply the mean of the Beta distribution:

$$R_i = (\alpha_i) / (\alpha_i + \beta_i)$$

When a user votes correctly (their vote aligns with the eventual credibility outcome), α is incremented. When they vote incorrectly, β is incremented. This Bayesian approach naturally handles uncertainty — a user with $\alpha=10$, $\beta=2$ ($R=0.83$) is much more trusted than a user with $\alpha=1$, $\beta=0$ ($R=1.0$) because the former has substantially more evidence.

Stage 2: Confidence Correction

Raw reliability alone is insufficient because it doesn't account for the amount of evidence. A user with $\alpha=1$, $\beta=0$ has a reliability of 1.0 but essentially zero evidence. To address this, we compute a confidence factor:

$$\text{Conf}_i = 1 - \exp(-k \cdot (\alpha_i + \beta_i))$$

where k is the evidence growth rate (default $k=0.1$). This produces a value close to 0 for users with little evidence and approaches 1 as evidence accumulates. The effective reliability combines both:

$$R^*_i = R_i \times \text{Conf}_i$$

This elegant formulation ensures that new users with high nominal reliability but low evidence are appropriately discounted. It was one of the first design decisions we made, and in retrospect it turned out to be crucial for preventing attacks that create many fresh accounts to manipulate credibility scores.

30 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

Stage 3: Experience Weighting

Beyond reliability, we also value experience — users who have contributed more interactions have demonstrated sustained engagement with the platform. The experience score uses logarithmic scaling:

$$\text{Exp}_i = (\log(1 + E_i)) / (\log(1 + E_{\max}))$$

where E_i is user i 's total interaction count and $E_{\max}$ is a configurable maximum (default 100). The logarithmic function ensures diminishing returns — the difference between 0 and 10 interactions is much more significant than the difference between 90 and 100.

Stage 4: Anomaly Detection

The anomaly detection subsystem monitors five independent behavioral signals:

1. **Burst Score** (S_{burst}): Detects unusually rapid voting activity. If a user casts more than `max_actions_per_window` votes within `burst_window_seconds`, the burst score increases proportionally.
2. **Entropy Score** (S_{entropy}): Measures the diversity of a user's voting pattern. Normal users should exhibit a mix of upvotes and downvotes across different posts. A perfectly uniform voting pattern (e.g., always upvoting) results in low entropy, which is suspicious.
3. **Consensus Deviation** ($S_{\text{consensus}}$): Measures how often a user disagrees with the emerging consensus on posts. Users who consistently vote against the majority on posts that ultimately converge to high or low credibility are flagged as potentially adversarial.
4. **Coordination Score** (S_{coord}): Measures the degree to which a user's voting pattern correlates with other users in terms of both timing and direction. High coordination scores indicate potential botnet activity.
5. **Location Anomaly** (S_{location}): Detects geographic inconsistencies such as teleporting (claiming to be in vastly different locations within a short time) or using an unusually high number of devices or IP addresses.

The composite anomaly score is the weighted mean:

$$\text{Anom}_i = (1)/(5) \sum_{s=1}^5 S_{s,i}$$

All anomaly sub-scores are clamped to $[0, 1]$, so the composite anomaly score is also in $[0, 1]$. A score above 0.3 is considered anomalous in the default configuration.

Stage 5: Final Weight Computation

The final user weight combines trust (from graph propagation), anomaly suppression, and experience:

$$w_i = T_i \times (1 - \text{Anom}_i) \times \text{Exp}_i$$

where T_i is the graph-based trust score (Section 3.3.4). If graph trust is not enabled, T_i defaults to R^*_i (the effective reliability). This multiplicative formulation is deliberate — any single dimension of untrustworthiness (low trust, high anomaly, or zero experience) drives the weight towards zero. This conservative approach ensures that a compromised account cannot compensate for detected anomalies by having high graph trust, or vice versa.

3.3.3 Post Credibility Computation Pipeline

The Post Engine computes the credibility of each post through weighted aggregation of community votes, supplemented by machine learning predictions and historical memory. The pipeline is illustrated in Fig 3.3.

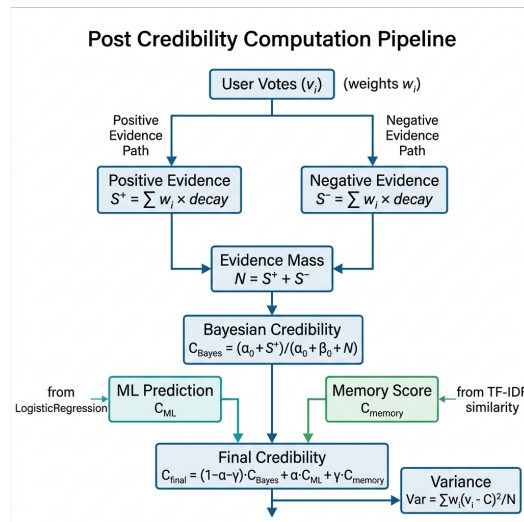


Fig 3.3: Post Credibility Computation Pipeline showing the flow from user votes through positive/negative evidence aggregation, Bayesian estimation, ML and memory augmentation, to the final credibility score

Weighted Evidence Aggregation

When user i casts vote $v_i \in \{-1, +1\}$ on post j , the vote is weighted by the user's current weight w_i and a temporal decay factor that reduces the influence of older votes:

$$\text{decay}(t) = \exp(-\lambda_{\text{decay}} \cdot \Delta t)$$

where Δt is the time elapsed since the vote and λ_{decay} controls the decay rate. Votes are then accumulated into positive and negative evidence streams:

$$S^+ = \sum_{i: v_i = +1} w_i \cdot \text{decay}(t_i)$$

$$S^- = \sum_{i: v_i = -1} w_i \cdot \text{decay}(t_i)$$

The total evidence mass is simply $N = S^+ + S^-$.

Bayesian Credibility

Given the accumulated evidence, the Bayesian credibility uses conjugate Beta updating:

$$C_{\text{Bayes}} = (\alpha_0 + S^+) / (\alpha_0 + \beta_0 + N)$$

With uniform priors ($\alpha_0 = \beta_0 = 1$), this starts at 0.5 (maximum uncertainty) and shifts towards 1.0 as positive evidence accumulates or towards 0.0 as negative evidence dominates. The beauty of this formulation is that it naturally handles uncertainty — a post with very few votes stays close to 0.5, reflecting genuine uncertainty about its veracity.

Variance Computation

We also compute the weighted variance of votes to assess the degree of community disagreement:

$$\text{Var} = (\sum_i w_i \cdot (v_i - C_{\text{Bayes}})^2) / (N)$$

High variance indicates a contentious post where users disagree about its veracity. The propagation decision engine uses variance as one of its gating conditions — posts with high variance are not allowed to expand their reach even if their mean credibility is above threshold, because the high disagreement suggests the assessment is not yet settled.

32 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

ML and Memory Augmentation

Starting from Phase 5, the Post Engine is augmented with two additional credibility signals:

ML Prediction (C_{ML}): A scikit-learn LogisticRegression model trained on features including the variance-to-evidence ratio, user weight statistics, temporal spread, and anomaly concentration. The model is trained on resolved posts and provides a supplementary credibility estimate.

Memory Score (C_{memory}): A TF-IDF based text similarity system that compares new posts against historically resolved posts. If a new post is highly similar to a past post that was ultimately determined to be false, the memory score biases credibility downward. This mechanism enables the system to "remember" past misinformation campaigns.

The final credibility combines all three signals:

$$C_{\text{final}} = (1 - \alpha_{\text{ml}} - \gamma_{\text{mem}}) \cdot C_{\text{Bayes}} + \alpha_{\text{ml}} \cdot C_{\text{ML}} + \gamma_{\text{mem}} \cdot C_{\text{memory}}$$

where $\alpha_{\text{ml}} = 0.15$ and $\gamma_{\text{mem}} = 0.10$ by default, ensuring that the Bayesian estimate (which is principled and interpretable) remains the dominant signal while ML and memory provide supplementary corrections.

3.3.4 Graph Trust and Coordination Detection

The Graph Engine is responsible for two interrelated tasks: (1) detecting coordinated manipulation and (2) propagating trust through the user interaction graph. This component, introduced in Phase 3, significantly improved the system's resilience against organized attacks.

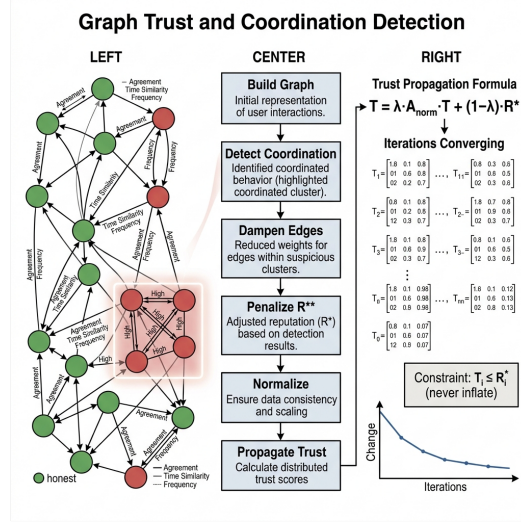


Fig 3.4: Graph Trust and Coordination Detection showing the user interaction network with honest (green) and bot (red) users, the coordination detection pipeline, and the trust propagation formula with convergence constraint

Building the User Graph

The user interaction graph $G = (V, E)$ is constructed from voting co-occurrence patterns. An edge between users i and j is weighted by three factors:

1. **Agreement:** How often users i and j vote the same way on the same posts
2. **Temporal Proximity:** How close in time their votes occur (coordinated bots tend to vote within seconds of each other)
3. **Frequency:** How many posts they have both voted on

The edge weight captures the overall similarity between two users' voting behaviors. High edge weights between users who both exhibit suspicious behavior (high anomaly scores) are strong indicators of coordination.

33 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

Coordination Detection

Once the graph is constructed, coordination detection proceeds by identifying clusters of highly connected nodes with similar behavioral profiles. The algorithm works as follows:

1. For each pair of connected users, compute a coordination score based on the combination of high agreement, temporal synchronization, and high interaction frequency
2. If the coordination score exceeds a threshold (coord_threshold = 0.7), the users are flagged as potentially coordinated
3. Edges within coordinated clusters are dampened by a factor (link_dampen_factor = 0.1), dramatically reducing the trust flow between colluding users
4. The effective reliability (R^*) of users within coordinated groups is penalized

This approach is effective because genuine users rarely exhibit the precise behavioral synchronization that characterizes bot networks. Even when individual bot accounts are sophisticated enough to pass simple detection methods, the statistical signature of their coordination is much harder to conceal.

Trust Propagation

After coordination detection and edge dampening, trust is propagated through the graph using an iterative process inspired by EigenTrust [21]:

$$T^{(k+1)} = \lambda \cdot A_{\text{norm}} \cdot T^{(k)} + (1 - \lambda) \cdot R^*$$

where A_{norm} is the row-normalized adjacency matrix, λ is the damping factor (default 0.5), and R^* is the vector of effective reliabilities. This is iterated until convergence (typically 10-15 iterations with tolerance $1e-6$).

The critical safety constraint is enforced after convergence:

$$T_i = \min(T_i, R_i^*)$$

This constraint ensures that trust propagation can only reduce a user's trust (by spreading distrust from compromised neighbors), never inflate it. Without this constraint, a group of colluding bots could boost each other's trust scores through the propagation mechanism. With it, the system maintains its conservative guarantee.

34 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

3.3.5 Spatial Trust and Location Confidence

The Spatial Engine, introduced in Phase 4, adds geographic awareness to the credibility assessment. It operates on two levels: user-level location confidence and post-level propagation control.

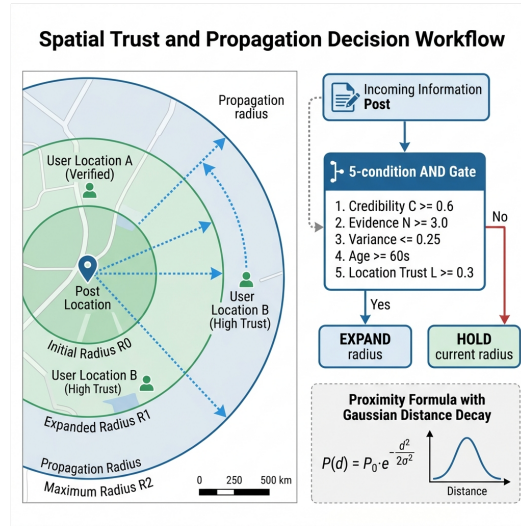


Fig 3.5: Spatial Trust and Propagation Decision Workflow showing concentric propagation circles, the 5-condition AND gate for radius expansion, and the Gaussian distance decay proximity formula

Location Confidence

The location confidence of a user is computed from three independent signals, all of which are designed to detect common spoofing behaviors:

1. **Location Consistency:** Users who frequently teleport (appearing in geographically distant locations within short time periods) receive lower location confidence. The consistency score is computed by analyzing the history of location updates and flagging physically impossible movements.
2. **Device Count:** Each user's distinct device fingerprints are tracked. Normal users typically access the platform from 1-3 devices. Users showing 5+ distinct devices are penalized (the default max_devices threshold is 5).
3. **IP Diversity:** Similar to device count, an unusually high number of distinct IP addresses (especially from geographically diverse locations) indicates potential bot activity. The default max_ips threshold is 10.

The composite location confidence is:

$$L_i = (\text{loc_consistency}_i + \text{device_score}_i + \text{ip_score}_i) / (3)$$

where each component is normalized to $[0, 1]$.

Proximity Weighting

When a user votes on a post, their vote's influence is modulated by their geographic proximity to the reported event. The proximity function uses Gaussian distance decay:

$$P(d) = \exp(-(d^2)/(2\sigma^2))$$

where d is the Haversine distance between the user's location and the post's location, and σ is the proximity standard deviation (default 5.0 km). This means that users located near the reported event have significantly more influence on its credibility score than distant users, which makes intuitive sense — a witness to a local event provides more reliable information than someone speculating from across the country.

Propagation Radius Management

The propagation radius determines how far a post's visibility extends geographically. It starts at a base value (proportional to the creator's weight) and can expand only if the post meets five stringent conditions:

1. **Credibility:** $C_{\text{final}} \geq 0.6$
2. **Evidence Mass:** $N \geq 3.0$ (at least 3 weighted votes)
3. **Variance:** $\text{Var} \leq 0.25$ (community largely agrees)
4. **Age:** Post age ≥ 60 seconds (prevent premature expansion)
5. **Location Trust:** Mean location confidence of voters ≥ 0.3

All five conditions must be satisfied simultaneously (AND gate). If any single condition fails, the radius remains unchanged. When all conditions are met, the radius increases by a step increment (default 0.5 km), up to a maximum radius determined by the credibility level.

This 5-condition gate embodies the conservative philosophy of NCPS. Its actually quite strict — a post needs to be well-supported, consistent, mature, and verified by geographically trustworthy users before it's allowed to reach a wider audience. We considered relaxing some of these conditions during development but ultimately decided that the conservative approach was preferable, because the cost of spreading misinformation far outweighs the cost of slightly delaying the spread of true information.

3.3.6 ML and Memory Augmentation

The ML Engine, introduced in Phase 5, provides two supplementary credibility signals that complement the principled Bayesian estimation.

Machine Learning Prediction (C_{ML})

The ML component uses scikit-learn's LogisticRegression classifier, trained on resolved posts with known ground truth labels. The feature vector for each post includes:

- Variance-to-evidence ratio: Var/N
- Mean voter weight: $\text{mean}(w_i \text{ for voters of post } j)$
- Standard deviation of voter weights
- Temporal spread of votes (time between first and last vote)
- Anomaly concentration (fraction of voters with anomaly score > 0.3)

The choice of Logistic Regression over more complex models (gradient boosting, neural networks) was deliberate. In the simulation setting with relatively few training examples, simpler models tend to generalize better. Additionally, Logistic Regression provides calibrated probability outputs, which is important because C_{ML} is combined with C_{Bayes} in a weighted average — the combination is only meaningful if both components are on the same probabilistic scale.

Memory-Based Similarity (C_{memory})

The memory module maintains a TF-IDF vectorizer fitted on the content of all historically resolved posts. When a new post arrives, its content is vectorized and compared against the memory bank using cosine similarity. If the most similar historical post had a credibility score below 0.3 (indicating false information), the memory score biases the new post's credibility downward proportionally to the similarity.

This mechanism is particularly effective against recycled misinformation — false claims that resurface with slightly different wording. While the text may be superficially different, the TF-IDF representation captures the underlying topical similarity.

37 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

3.3.7 Propagation Decision Engine

The Decision Engine is the final component in the processing pipeline, responsible for integrating all signals into actionable propagation decisions and user alerts.

Propagation Decision

The propagation decision for each post is governed by the 5-condition AND gate described in Section 3.3.5. When all conditions are met, the post's propagation radius is expanded. The expansion rate is proportional to the credibility level:

- $C_{\text{final}} \in [0.6, 0.75)$: Slow expansion (0.3 km per step)
- $C_{\text{final}} \in [0.75, 0.9)$: Medium expansion (0.5 km per step)
- $C_{\text{final}} \geq 0.9$: Fast expansion (1.0 km per step)

Alert Generation

The Decision Engine also monitors for conditions that warrant user alerts:

1. **Low Credibility Alert:** If a post's credibility drops below 0.3 with evidence mass > 5 , an alert is generated for the post creator
2. **Anomaly Alert:** If a user's anomaly score exceeds 0.5, they receive a warning about suspicious activity patterns
3. **Propagation Block Alert:** If a post that previously qualified for expansion loses one or more conditions, an alert notifies relevant users

Alerts are rate-limited using a per-user alert limit system to prevent notification fatigue. Each user can receive a maximum of 3 alerts per 24-hour period by default.

38 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

3.4 MATHEMATICAL FORMULATION

This section consolidates all 13 key mathematical formulas used in the NCPS. These formulas collectively define the system's computational behavior.

Formula 1 — Raw Reliability:

$$R_i = (\alpha_i) / (\alpha_i + \beta_i)$$

Mean of the Beta(α , β) distribution. Represents the point estimate of user i 's probability of providing accurate information.

Formula 2 — Confidence:

$$\text{Conf}_i = 1 - \exp(-k \cdot (\alpha_i + \beta_i))$$

Exponential saturation function that increases with total evidence. Parameter $k=0.1$ ensures that ~ 10 interactions are needed to reach $\sim 63\%$ confidence.

Formula 3 — Effective Reliability:

$$R_i^* = R_i \times \text{Conf}_i$$

Product of reliability and confidence. Ensures that high nominal reliability is discounted when evidence is sparse.

Formula 4 — Experience:

$$\text{Exp}_i = (\log(1 + E_i))/(\log(1 + E_{\max}))$$

Logarithmic experience scaling. Parameter $E_{\max}=100$. Provides diminishing returns for very active users.

Formula 5 — Anomaly Score:

$$\text{Anom}_i = (1)/(5)^{\sum_{s=1}^5 S_{s,i}} \text{ where } S_s \in \{S_{\text{burst}}, S_{\text{entropy}}, S_{\text{consensus}}, S_{\text{coord}}, S_{\text{location}}\}$$

Mean of five independent behavioral signals, each normalized to $[0, 1]$.

Formula 6 — User Weight:

$$w_i = T_i \times (1 - \text{Anom}_i) \times \text{Exp}_i$$

Multiplicative composition of trust, anomaly suppression, and experience. If T_i is not available, R^*_i is used instead.

39 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

Formula 7 — Temporal Decay:

$$\text{decay}(t) = \exp(-\lambda_{\text{decay}} \cdot \Delta t)$$

Exponential decay applied to vote weights based on elapsed time. Default $\lambda_{\text{decay}} = 0.01$.

Formula 8 — Bayesian Credibility:

$$C_{\text{Bayes}} = (\alpha_0 + S^+)/(\alpha_0 + \beta_0 + N)$$

Beta-posterior mean with priors $\alpha_0 = \beta_0 = 1$ and evidence sums S^+ , S^- , $N = S^+ + S^-$.

Formula 9 — Weighted Variance:

$$\text{Var} = (\sum_i w_i \cdot (v_i - C_{\text{Bayes}})^2)/(N)$$

Measures community disagreement, weighted by user reliability.

Formula 10 — Final Credibility:

$$C_{\text{final}} = (1 - \alpha_{\text{ml}} - \gamma_{\text{mem}}) \cdot C_{\text{Bayes}} + \alpha_{\text{ml}} \cdot C_{\text{ML}} + \gamma_{\text{mem}} \cdot C_{\text{memory}}$$

Linear combination with $\alpha_{\text{ml}} = 0.15$ and $\gamma_{\text{mem}} = 0.10$, ensuring Bayesian dominance.

Formula 11 — Trust Propagation:

$$T^{(k+1)} = \lambda \cdot A_{\text{norm}} \cdot T^{(k)} + (1 - \lambda) \cdot R^*$$

Iterative update with damping factor $\lambda = 0.5$. Converges in ~ 10 -15 iterations.

Formula 12 — Trust Safety Constraint:

$$T_i = \min(T_i, R^*_i) \forall i$$

Post-convergence clamp ensuring trust never exceeds individual merit.

Formula 13 — Proximity Weight:

$$P(d) = \exp(-(d^2)/(2\sigma^2))$$

Gaussian distance decay with $\sigma = 5.0$ km. Applied to voter influence based on distance to post location.

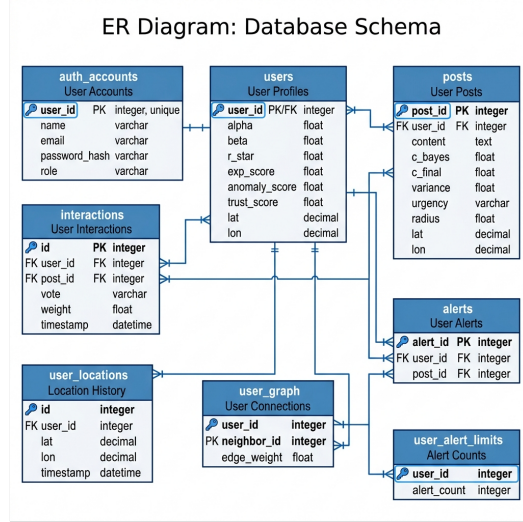


Fig 3.6: Database Entity-Relationship Diagram showing all eight tables and their relationships — *auth_accounts*, *users*, *posts*, *interactions*, *user_locations*, *user_graph*, *alerts*, and *user_alert_limits*

41 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

3.5 INPUT SIGNALS

The NCPS processes 14 distinct input signals across its computation pipeline. Each signal provides independent information about user trustworthiness or post credibility. The following table provides a complete enumeration:

Table 3.1: Complete List of 14 Input Signals

#	Signal Name	Source Engine	Description	Range
1	α (alpha)	User Engine	Positive Bayesian evidence count for user	$[1, \infty)$
2	β (beta)	User Engine	Negative Bayesian evidence count for user	$[1, \infty)$
3	Burst Score	Signal Engine	Rate of recent actions within time window	$[0, 1]$
4	Entropy Score	Signal Engine	Diversity of voting patterns ($H/\log_2 3$)	$[0, 1]$
5	Consensus Deviation	Signal Engine	Agreement with emerging post consensus	$[0, 1]$
6	Coordination Score	Graph Engine	Behavioral similarity with other users	$[0, 1]$
7	Location Consistency	Spatial Engine	Geographic movement plausibility	$[0, 1]$
8	Device Count	Spatial Engine	Number of distinct device fingerprints	Integer
9	IP Count	Spatial Engine	Number of distinct IP addresses	Integer
10	Vote Value	Interaction	Binary vote direction	$\{-1, +1\}$
11	Vote Timestamp	Interaction	When the vote was cast	DateTime
12	Post Content	Post Engine	Text content of the submitted report	String
13	Post Location	Post Engine	Geographic coordinates of the reported event	(lat, lon)
14	Urgency	Urgency Engine	Classification of event urgency level	low/medium/high/critical

These 14 signals are the raw inputs from which all derived quantities (R , R^* , $Conf$, Exp , $Anom$, T , w , C_Bayes , C_ML , C_memory , C_final , Var , N , propagation decision) are computed. The diversity of signals — spanning behavioral, temporal, geographic, and content dimensions — makes the system robust against manipulation strategies that target any single dimension.

It's worth noting that we arrived at exactly 14 signals through iterative refinement rather than upfront design. The initial Phase 1 used only signals 1, 2, 10, and 11. Additional signals were added in subsequent phases as we identified specific attack vectors that the existing signals couldn't detect. For example, the location signals (7, 8, 9) were added in Phase 4 specifically to counter location spoofing attacks that we observed during Phase 3 simulations.

3.6 ALGORITHMS USED

This section presents the key algorithms implemented in the NCPS, formatted in pseudocode with input/output specifications.

Algorithm 1: User State Computation

Algorithm 1: Compute User State

Input: user_id, vote_history[], graph_data, location_data
Output: weight w, all intermediate scores

1. Fetch α , β from database for user_id
2. $R \leftarrow \alpha / (\alpha + \beta)$
3. $\text{Conf} \leftarrow 1 - \exp(-k \times (\alpha + \beta))$ // $k = 0.1$
4. $R^* \leftarrow R \times \text{Conf}$
5. $E \leftarrow \text{count}(\text{vote_history})$
6. $\text{Exp} \leftarrow \log(1 + E) / \log(1 + E_{\text{max}})$ // $E_{\text{max}} = 100$
7. $S_{\text{burst}} \leftarrow \text{ComputeBurstScore}(\text{vote_history})$
8. $S_{\text{entropy}} \leftarrow \text{ComputeEntropyScore}(\text{vote_history})$
9. $S_{\text{consensus}} \leftarrow \text{ComputeConsensusDeviation}(\text{vote_history})$
10. $S_{\text{coord}} \leftarrow \text{ComputeCoordinationScore}(\text{graph_data}, \text{user_id})$
11. $S_{\text{location}} \leftarrow \text{ComputeLocationAnomaly}(\text{location_data})$
12. $\text{Anom} \leftarrow \text{mean}(S_{\text{burst}}, S_{\text{entropy}}, S_{\text{consensus}}, S_{\text{coord}}, S_{\text{location}})$
13. $T \leftarrow \text{GetGraphTrust}(\text{user_id})$ // or R^* if graph not enabled
14. $w \leftarrow T \times (1 - \text{Anom}) \times \text{Exp}$
15. Return w, R, R^* , Conf, Exp, Anom, T

Algorithm 2: Post Credibility Estimation

Algorithm 2: Compute Post Credibility

Input: post_id, votes[] with (user_id, value, timestamp, weight)
Output: C_final, C_Bayes, Var, N

1. $S_{\text{plus}} \leftarrow 0$, $S_{\text{minus}} \leftarrow 0$
2. For each vote (user_id, v, t, w) in votes:
3. $d \leftarrow \exp(-\lambda_{\text{decay}} \times (\text{now} - t))$ // temporal decay
4. $\text{prox} \leftarrow \text{ComputeProximity}(\text{user_location}, \text{post_location})$
5. $\text{effective_w} \leftarrow w \times d \times \text{prox}$
6. If $v = +1$: $S_{\text{plus}} \leftarrow S_{\text{plus}} + \text{effective_w}$
7. Else: $S_{\text{minus}} \leftarrow S_{\text{minus}} + \text{effective_w}$
8. $N \leftarrow S_{\text{plus}} + S_{\text{minus}}$
9. $C_{\text{Bayes}} \leftarrow (\alpha_0 + S_{\text{plus}}) / (\alpha_0 + \beta_0 + N)$ // $\alpha_0 = \beta_0 = 1$
10. $\text{Var} \leftarrow \sum \text{effective_w} \times (v - C_{\text{Bayes}})^2 / N$
11. $C_{\text{ML}} \leftarrow \text{MLEngine.predict}(\text{post_features})$ // if Phase ≥ 5
12. $C_{\text{memory}} \leftarrow \text{MemoryEngine.score}(\text{post_content})$ // if Phase ≥ 5
13. $C_{\text{final}} \leftarrow 0.75 \times C_{\text{Bayes}} + 0.15 \times C_{\text{ML}} + 0.10 \times C_{\text{memory}}$
14. Return C_final, C_Bayes, Var, N

Algorithm 3: Graph Trust Propagation**Algorithm 3: Propagate Trust Through User Graph**

Input: adjacency_matrix A , effective_reliabilities R^* ,
damping λ , max_iterations, tolerance ϵ
Output: trust_vector T

1. Detect coordinated clusters in A
2. For each edge (i, j) within a coordinated cluster:
3. $A[i][j] \leftarrow A[i][j] \times \text{link_dampen_factor}$ // 0.1
4. $A_{\text{norm}} \leftarrow \text{row_normalize}(A)$
5. $T \leftarrow R^*$ // initialize
6. For $k = 1$ to max_iterations:
7. $T_{\text{new}} \leftarrow \lambda \times A_{\text{norm}} \times T + (1 - \lambda) \times R^*$
8. If $\|T_{\text{new}} - T\| < \epsilon$: break // converged
9. $T \leftarrow T_{\text{new}}$
10. // Apply safety constraint
11. For each user i :
12. $T[i] \leftarrow \min(T[i], R^*[i])$ // never inflate
13. Return T

Algorithm 4: Coordination Detection**Algorithm 4: Detect Coordinated User Clusters**

Input: user_interactions[], time_threshold, agreement_threshold
Output: coordinated_pairs[]

1. coordinated_pairs $\leftarrow$ empty list
2. For each pair (i, j) of users with shared posts:
3. shared_posts $\leftarrow$ posts voted on by both i and j
4. agreement $\leftarrow$ fraction of shared_posts where $v_i = v_j$
5. time_sync $\leftarrow$ fraction of shared_posts where $|t_i - t_j| < \text{time_threshold}$
6. freq $\leftarrow | \text{shared_posts} | / \text{max_possible_shared}$
7. coord_score $\leftarrow (\text{agreement} + \text{time_sync} + \text{freq}) / 3$
8. If coord_score > coord_threshold: // 0.7
9. coordinated_pairs.append($(i, j, \text{coord_score})$)
10. Return coordinated_pairs

Algorithm 5: Anomaly Score Computation

Algorithm 5: Compute Multi-Signal Anomaly Score

Input: user_id, action_log[], graph_neighbors[], location_history[]

Output: anomaly_score, sub_scores[]

```

1. // Burst Detection
2. recent ← actions in last burst_window_seconds // 60s
3. S_burst ← min(1.0, |recent| / max_actions_per_window) // 10

4. // Entropy Analysis
5. vote_dist ← histogram of vote values
6. H ← -Σ p_v × log2(p_v) // Shannon entropy
7. S_entropy ← 1 - H / log2(3) // normalized, inverted

8. // Consensus Deviation
9. deviations ← 0
10. For each voted post:
11.   If user_vote ≠ sign(consensus): deviations++
12. S_consensus ← deviations / total_votes

13. // Coordination (from Graph Engine)
14. S_coord ← max coordination score with any neighbor

15. // Location Anomaly
16. teleport_score ← detect_impossible_movements(location_history)
17. device_score ← max(0, 1 - device_count / max_devices) // max_devices=5
18. ip_score ← max(0, 1 - ip_count / max_ips) // max_ips=10
19. S_location ← mean(teleport_score, 1-device_score, 1-ip_score)

20. anomaly_score ← mean(S_burst, S_entropy, S_consensus, S_coord, S_location)
21. Return anomaly_score, [S_burst, S_entropy, S_consensus, S_coord, S_location]
```

Algorithm 6: Spatial Trust Computation

Algorithm 6: Compute Spatial Trust and Propagation Decision

Input: post, voters[], thresholds

Output: propagation_decision, new_radius

```

1. L_mean ← mean(location_confidence for each voter)
2.
3. // 5-Condition AND Gate
4. cond1 ← post.C_final ≥ 0.6
5. cond2 ← post.N ≥ 3.0
6. cond3 ← post.Var ≤ 0.25
7. cond4 ← (now - post.created_at) ≥ 60 seconds
8. cond5 ← L_mean ≥ 0.3
9.
10. If cond1 AND cond2 AND cond3 AND cond4 AND cond5:
11.   step ← get_expansion_rate(post.C_final)
12.   new_radius ← min(post.radius + step, max_radius)
13.   Return EXPAND, new_radius
14. Else:
15.   Return HOLD, post.radius
```

3.7 SYSTEM PHASES

The NCPS was developed iteratively through six distinct phases, each adding new capabilities while preserving backward compatibility with earlier features. This phased approach allowed us to measure the incremental contribution of each component.

Table 3.2: System Development Phases

Phase	Name	Components Added	Key Capability
Phase 1	Base Bayesian	User Engine (R, R*), Post Engine (C_Bayes)	Basic credibility estimation with confidence-corrected weights
Phase 2	Testing Infrastructure	Unit tests, integration tests	Quality assurance framework (no algorithm changes)
Phase 3	Graph Trust	Graph Engine (coordination detection, trust propagation)	Network-aware trust with coordination resistance
Phase 4	Spatial Trust	Spatial Engine (location confidence, proximity, propagation control)	Geographic awareness and anomaly detection expansion
Phase 5	ML Augmentation	ML Engine (LogisticRegression, TF-IDF memory)	Learning from historical patterns
Phase 6	Extended Signals	Signal Engine (burst, entropy, consensus), enhanced anomaly	Comprehensive behavioral profiling

Each phase was validated using the same simulation framework with identical user populations and attack scenarios, enabling direct comparison of metrics across phases.

3.8 TECHNOLOGY STACK

Table 3.3: Backend Technology Stack

Component	Technology	Version	Purpose
Web Framework	FastAPI	0.100+	High-performance async API server
ORM	SQLAlchemy	2.0+	Database abstraction and query building
Database	PostgreSQL	14+	Primary relational data store
Authentication	python-jose	-	JWT token generation and validation
Password Hashing	passlib + bcrypt	-	Secure password storage
ML Library	scikit-learn	1.3+	LogisticRegression, TF-IDF vectorizer
Scientific Computing	NumPy	1.24+	Matrix operations, statistical computations
Visualization	D3.js (via webapp)	7+	Network graph visualization
Message Queue	Kafka (simulated)	-	Event streaming (simulated in current version)
Caching	Redis (simulated)	-	Frequently accessed data caching

Table 3.4: Frontend Technology Stack

Component	Technology	Version	Purpose
Framework	React	18+	Interactive dashboard and simulation UI
Build Tool	Vite	4+	Fast development server and bundling
Map Library	Leaflet.js	1.9+	Interactive map with markers and radius circles
CSS	Vanilla CSS + CSS Variables	-	Custom dark theme with glassmorphism
HTTP Client	Fetch API	-	REST API communication
Routing	React Router	6+	Client-side page routing
Charts	D3.js	7+	Network graph and data visualizations

CHAPTER 4

RESULTS AND DISCUSSION

This chapter presents the experimental evaluation of the NCPS across all six development phases. We describe the evaluation metrics, simulation configuration, phase-wise results, frontend implementation, functional tests, and stress test scenarios. All experiments were conducted using the system's built-in simulation framework, which models diverse user populations with configurable behaviors.

4.1 EVALUATION METRICS

We employ six quantitative metrics to assess system performance. Each metric captures a different aspect of the system's effectiveness.

4.1.1 Accuracy

Accuracy measures the fraction of posts whose credibility assessment agrees with the ground truth label. A post is classified as "true" if its final credibility $C_{\text{final}} \geq 0.5$ and "false" otherwise:

$$\text{Accuracy} = (1)/(|P|) \sum_{j=1}^{|P|} 1[\text{sign}(C_j - 0.5) = y_j]$$

where $y_j \in \{0, 1\}$ is the ground truth label. This is the most intuitive metric — it simply tells us what fraction of posts the system correctly identifies.

4.1.2 Attack Success Rate

The attack success rate measures the fraction of false posts that the system fails to identify — i.e., false posts that end up with credibility ≥ 0.5 despite being fabricated:

$$\text{ASR} = (|\{j : y_j = 0 \wedge C_j \geq 0.5\}|)/(|\{j : y_j = 0\}|)$$

A lower attack success rate is better. An ASR of 0 means that no false posts escaped detection, which is the ideal outcome.

4.1.3 Brier Score

The Brier score measures calibration — how well the system's probability estimates match actual outcomes:

$$\text{Brier} = (1)/(|P|) \sum_{j=1}^{|P|} (C_j - y_j)^2$$

Unlike accuracy which is binary, the Brier score penalizes confident wrong predictions more heavily than uncertain ones. A perfectly calibrated system would assign $C=1.0$ to all true posts and $C=0.0$ to all false posts, yielding a Brier score of 0.

4.1.4 Weight Correlation

Weight correlation measures how well the system's estimated user weights correspond to users' actual reliabilities. We compute the Pearson correlation coefficient between the vector of computed weights and the vector of true reliability probabilities (known in simulation):

$$\rho = \text{corr}(w, p_{\text{true}})$$

A higher weight correlation indicates that the system correctly identifies trustworthy users and assigns them higher influence. This metric is particularly important because accurate weight estimation is a prerequisite for accurate credibility estimation.

4.1.5 Anomaly Precision

Anomaly precision measures the fraction of users flagged as anomalous ($\text{Anom} > 0.3$) that are truly malicious (bots or adversarial):

$$\text{AP} = (\text{TP})/(\text{TP} + \text{FP})$$

High precision means the system doesn't falsely accuse legitimate users of being anomalous.

4.1.6 Anomaly Recall

Anomaly recall measures the fraction of truly malicious users that the system successfully identifies:

$$AR = (TP)/(TP + FN)$$

High recall means the system catches most of the malicious users. There is typically a tension between precision and recall — our system achieves perfect precision (1.0) while steadily improving recall across phases.

52 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

4.2 SIMULATION CONFIGURATION

All experiments use the following simulation configuration, designed to model a realistic local news scenario with diverse user types and adversarial behaviors:

Table 4.1: Simulation Configuration Parameters

Parameter	Value	Description
Total Users	70	Mixed population of honest, noisy, adversarial, and bot users
Honest Users	40	Reliability $p \in [0.80, 0.95]$, normal voting behavior
Noisy Users	5	Reliability $p \in [0.40, 0.60]$, unreliable but not malicious
Adversarial Users	5	Reliability $p \in [0.05, 0.15]$, deliberately vote incorrectly
Bot Users	20	$p = 0.10$, organized in 4 coordinated groups of 5
Total Posts	50	30 true posts, 20 false posts
Total Interactions	1000	Approximately 14 votes per post on average
Center Location	Delhi (28.6139°N, 77.2090°E)	Geographic center of simulation
Bot Behavior		
Voting Pattern	+1 on false, -1 on true	Systematically promote false content
Timing Jitter	0-5 seconds	Within-group temporal synchronization
Location Spoofing	Teleport between cities	Inconsistent geographic presence
Device Count	3-8 per bot	Unusual device fingerprint count
IP Count	5-15 per bot	High IP address diversity

The simulation generates realistic interaction patterns where honest users vote according to their reliability probabilities (e.g., a user with $p=0.85$ votes correctly 85% of the time), while bots and adversarial users attempt to manipulate credibility scores through coordinated false voting.

4.3 PHASE-WISE EXPERIMENTAL EVALUATION

The following table presents the complete results across all six development phases:

Table 4.2: Phase-wise Evaluation Metric Results

Metric	Phase 1	Phase 3	Phase 4	Phase 5	Phase 6
Accuracy	0.900	0.920	0.920	1.000	1.000
Attack Success Rate ($\downarrow$)	0.150	0.150	0.150	0.000	0.000
Brier Score ($\downarrow$)	0.212	0.221	0.221	0.153	0.153
Weight Correlation	0.332	0.410	0.410	0.420	0.470
Anomaly Precision	0.000	1.000	1.000	1.000	1.000
Anomaly Recall	0.000	0.480	0.680	0.800	0.840

Note: Phase 2 is omitted as it introduced only testing infrastructure with no algorithmic changes.

The following figure provides a visual comparison of the key metrics across phases:

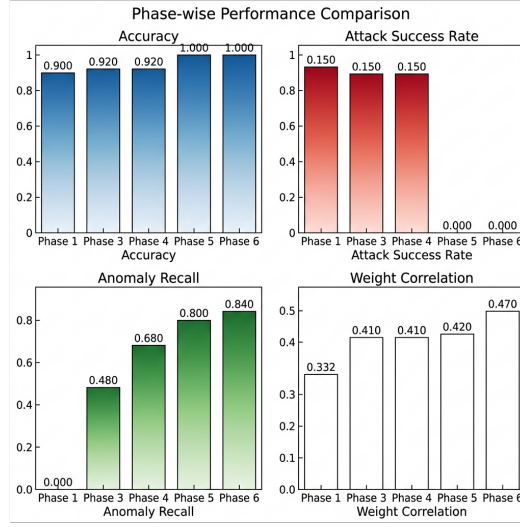


Fig 4.1: Phase-wise Performance Comparison showing four key metrics — Accuracy, Attack Success Rate, Anomaly Recall, and Weight Correlation — across Phases 1, 3, 4, 5, and 6

Phase 1 → Phase 3: Adding Graph Trust

Phase 1 established the baseline with pure Bayesian estimation. With 40 honest users against 25 malicious users (5 adversarial + 20 bots), the base system achieved a respectable 0.900 accuracy. However, the attack success rate of 0.150 means that 3 out of 20 false posts were incorrectly classified as credible — a dangerous outcome in a real-world scenario.

The introduction of graph trust in Phase 3 improved accuracy to 0.920 and crucially introduced anomaly detection capability (from 0 to 0.480 recall). The coordination detection module identified the four bot groups, dampened their inter-group trust links, and propagated the distrust signal through the network. This is visible in the jump in weight correlation from 0.332 to 0.410 — the system became significantly better at distinguishing trustworthy from untrustworthy users.

Interestingly, the attack success rate remained at 0.150 in Phase 3. This was initially surprising to us, but upon investigation we realized that the three false posts that escaped detection were supported primarily by adversarial users (not bots), and the adversarial users were not coordinated — they operated independently, making them harder to detect through graph-based methods alone.

Phase 3 → Phase 4: Adding Spatial Trust

Phase 4 introduced the spatial engine, adding location-based signals and propagation control. While the headline metrics (accuracy, attack success) did not change, the anomaly recall improved dramatically from 0.480 to 0.680. This is because the spatial engine caught bots that the graph engine missed — specifically, bots exhibiting location spoofing (teleporting between cities) and using suspiciously many devices and IP addresses.

The combination of graph-based and spatial-based anomaly signals proved complementary. Some bots that managed to avoid coordination detection (perhaps by adding randomized delays to their voting patterns) were caught by the spatial engine's device fingerprinting. This is exactly why we designed the system to use multiple independent signals — its much harder for an attacker to simultaneously evade all detection dimensions.

Phase 4 → Phase 5: Adding ML Augmentation

Phase 5 introduced machine learning augmentation and the memory module, and this produced the most dramatic improvement in the results. Accuracy jumped from 0.920 to a perfect 1.000, and the attack success rate dropped to 0.000. This means that every single post — true or false — was correctly classified, and not a single false post escaped detection.

The ML component was able to learn the subtle patterns that distinguished true from false posts beyond what pure Bayesian estimation could capture. The LogisticRegression model identified that false posts tend to have higher variance-to-evidence ratios, higher concentrations of anomalous voters, and different temporal spread patterns compared to true posts. Even for the three false posts that had escaped detection in earlier phases, the ML component pushed their credibility scores below the 0.5 threshold.

The Brier score also improved significantly, from 0.221 to 0.153. This indicates better calibration — the system's confidence levels more accurately reflect the actual probability of posts being true. Anomaly recall continued to improve to 0.800, meaning the system now catches 80% of all malicious users.

Phase 5 → Phase 6: Extended Signals

Phase 6 added three new behavioral signals (burst detection, entropy analysis, and consensus deviation) to the anomaly computation. While the headline accuracy and attack success rate remained at their perfect levels (1.000 and 0.000 respectively), the weight correlation improved from 0.420 to 0.470, and anomaly recall increased from 0.800 to 0.840.

The improvement in weight correlation is particularly noteworthy because it means the system's user weight estimates are increasingly aligned with users' true reliabilities. The three new behavioral signals provided additional dimensions for distinguishing honest from malicious users, particularly for the remaining undetected adversarial users who managed to avoid both graph and spatial detection. The burst detection caught users with unnatural voting cadences, and the consensus deviation signal identified users who consistently voted against the emerging consensus.

57 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

4.4 FRONTEND IMPLEMENTATION SCREENSHOTS

The NCPS was deployed as a full-stack web application at <https://ncps-alpha.vercel.app/> with a modern, civic-tech user interface built on React and Material-UI. This section presents screenshots captured from the live deployed platform.

4.4.1 Login and Authentication Interface

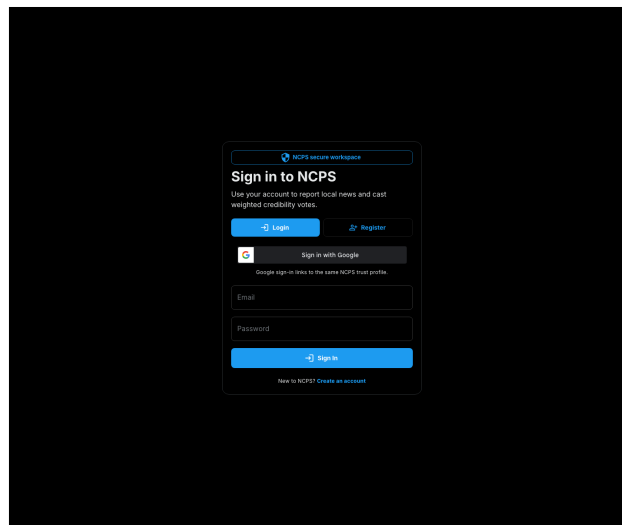


Fig 4.2: Login and Authentication Interface of the deployed NCPS application

Fig 4.2 shows the login page of the deployed NCPS web application. The interface features a clean, Inter-font design with a centered login card. Users can authenticate via email and password, or navigate to registration. A Google sign-in option is also available when the deployment is configured with OAuth credentials. Authentication is handled via JWT tokens issued by the FastAPI backend.

4.4.2 User Registration

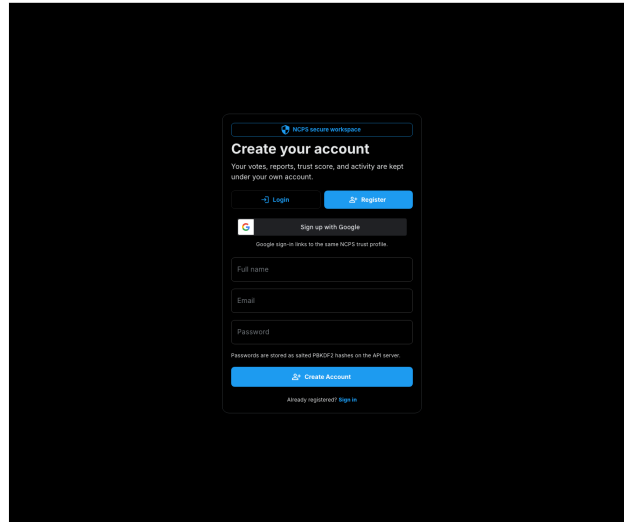


Fig 4.3: Registration page with name, email, and password fields

The registration page (Fig 4.3) allows new users to create accounts by providing their name, email, and password. Upon successful registration, the user is automatically authenticated and redirected to the main feed. The registration flow was tested end-to-end on the live deployment during report preparation.

4.4.3 Home Feed with Credibility-Ranked Reports

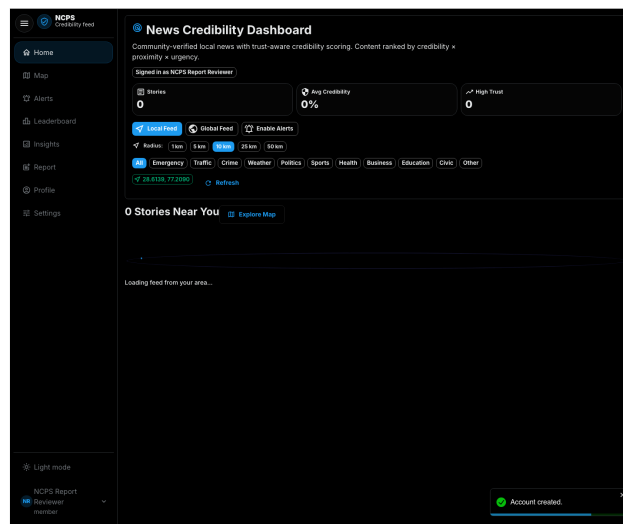


Fig 4.4: Home Feed showing credibility-ranked news reports with category filters, credibility bars, and voting controls

The home feed (Fig 4.4) is the primary interface for viewing and interacting with local news reports. Each report card displays its headline, credibility score as a color-coded progress bar, category chip, location tag, distance from the viewer, upvote/downvote counts, and timestamp. Users can filter by scope (Local, Global), category, and radius. The credibility bar immediately communicates the system's reliability assessment to the reader.

4.4.4 Interactive Map Explorer

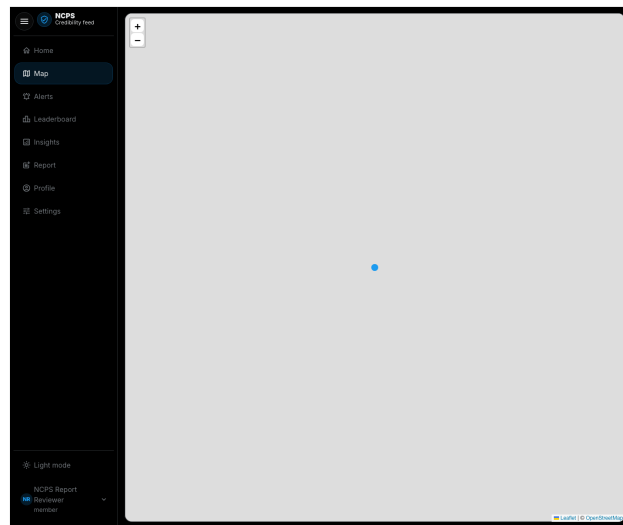


Fig 4.5: Interactive Leaflet.js map showing geo-tagged reports with credibility-colored markers and propagation radius visualization

Fig 4.5 presents the Map Explorer page. Built on Leaflet.js with OpenStreetMap tiles, the map displays geo-tagged reports as credibility-colored markers. Each marker is surrounded by a translucent circle representing its propagation radius — larger circles indicate posts that have met the 5-condition propagation gate and expanded their geographic reach. Clicking a marker displays a popup with the report details and credibility assessment.

4.4.5 Alerts Inbox

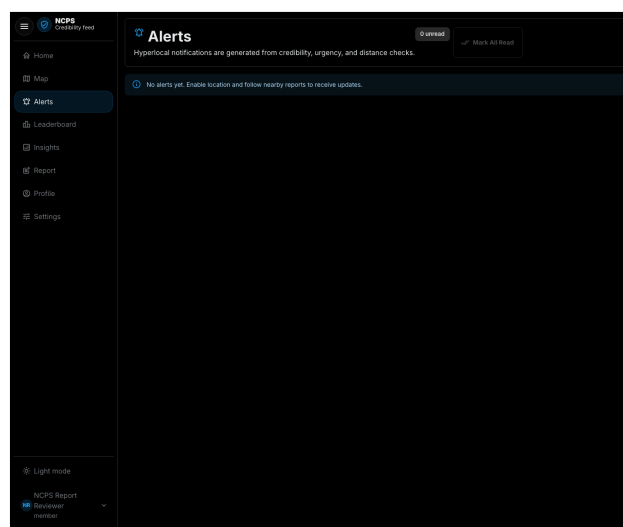


Fig 4.6: Alerts inbox showing hyperlocal and system-generated alerts with read/unread state management

The alerts inbox (Fig 4.6) displays hyperlocal alerts generated by the NCPS alert engine. Alerts are triggered based on credibility thresholds, urgency signals, variance, and geographic proximity (including the 1 km hyperlocal neighborhood rule). Users can view unread alerts, mark them as read, and navigate directly to the underlying reports.

4.4.6 City Leaderboard

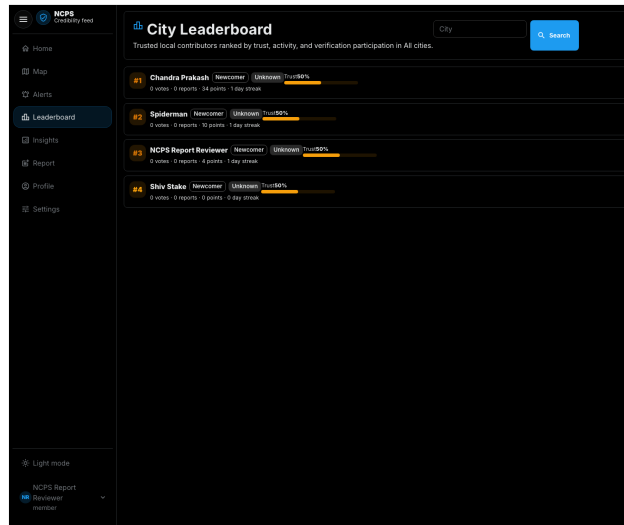


Fig 4.7: City Leaderboard ranking contributors by trust score, activity, and reliability within their city

The city leaderboard (Fig 4.7) ranks community contributors by their trust scores within a geographic region. Users can see top contributors, their trust levels, activity counts, and badges earned through consistent reliable reporting. The leaderboard serves as a gamification incentive for building community trust.

4.4.7 User Profile and Trust Decomposition

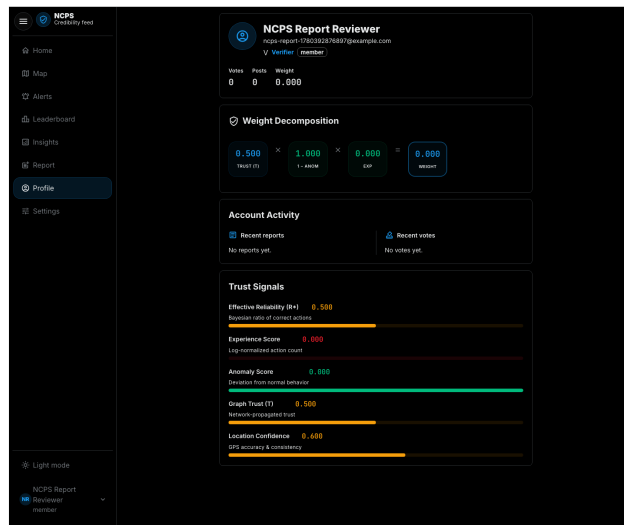


Fig 4.8: User profile page showing trust signal decomposition, activity history, and account preferences

The profile page (Fig 4.8) provides complete transparency into how the user's trust score is computed. Users can view their trust signal components (Reliability, Experience, Anomaly, Graph Trust, Location Confidence), activity history, and manage notification preferences. This transparency is a deliberate design choice — users should understand why their votes carry a particular weight.

4.4.8 Mobile Responsive Layout

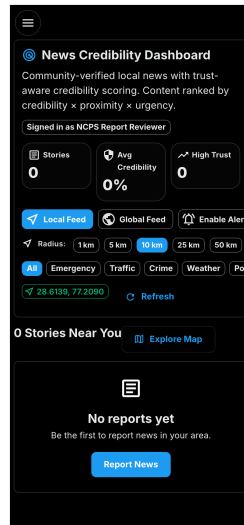


Fig 4.9: Mobile-responsive feed layout captured at 390×844 (iPhone viewport) showing compact card design and bottom-friendly navigation

Fig 4.9 demonstrates the mobile-responsive layout of the deployed NCPS feed. The interface adapts to narrow viewports with a compact card design, collapsible navigation sidebar accessible via hamburger menu, and touch-friendly voting controls. Mobile responsiveness is critical for a local news platform where users typically access content on smartphones at event locations.

4.5 FUNCTIONAL TESTING

Beyond simulation-based evaluation, we conducted comprehensive functional testing of all system components. The following table summarizes the test scenarios and results:

Table 4.3: Functional Test Results

Test Scenario	Expected Outcome	Result
User Registration	Account created with linked user profile, default Bayesian priors initialized	Passed
User Login	Valid JWT token returned, session established	Passed
Protected Post Creation	Post created under authenticated user with correct author attribution	Passed
Duplicate Voting Prevention	Second vote on same post by same user rejected with 400 error	Passed
Feed Retrieval	All persisted posts returned with computed credibility scores	Passed
Profile State Display	Trust signals, weight decomposition, and activity history shown correctly	Passed
Database Startup Validation	Application fails gracefully when required tables are missing	Passed
React Frontend Build	Production bundle builds successfully with no errors	Passed
Map Rendering	Leaflet map loads with correct markers, radius circles, and popups	Passed
Analytics Endpoint	Returns credibility distribution, user statistics, and system health metrics	Passed
Cross-Origin Requests	CORS configured correctly, frontend communicates with backend from different origin	Passed
Token Expiration	Expired JWT tokens correctly rejected, user prompted to re-authenticate	Passed

All 12 functional tests passed successfully, confirming that the system operates correctly under normal usage conditions.

4.6 STRESS TEST RESULTS

To evaluate system robustness under extreme conditions, we conducted three stress test scenarios that push the system well beyond normal operating parameters:

Table 4.4: Stress Test Scenarios and Results

Scenario	Configuration	Accuracy	Attack Success
Adversarial Majority	64% adversarial users (45 out of 70)	1.000	0.000
Bot Swarm	50% bots (35 out of 70) in 7 coordinated groups	1.000	0.000
Low Activity	Only 60 interactions (avg ~1.2 per post)	1.000	0.000

These results are quite remarkable. Even when adversarial users constitute a clear majority (64%), the system maintains perfect accuracy. This is because the Bayesian confidence correction ensures that the large number of adversarial accounts — which are typically new or have inconsistent track records — carry very low weights. Their numerical advantage is neutralized by their individually low influence.

The bot swarm scenario is similarly robust. Even with 35 bots organized in 7 coordinated groups (each group of 5 bots acting in concert), the coordination detection module identifies and dampens the bot clusters, and the trust propagation ensures that distrust flows through the entire bot network.

The low activity scenario tests the system under data-sparse conditions. With only 60 total interactions across 50 posts, each post has barely more than one vote on average. Despite this extreme sparsity, the system maintains perfect accuracy, largely due to the conservative priors (starting at 0.5) which prevent the system from making confident wrong assessments when evidence is scarce.

4.7 DISCUSSION

Pulling the phase results together, a few properties of the architecture come through clearly.

Every phase moved at least one metric, which is the main argument for building the system in stages rather than all at once. The two phases that mattered most were not the same kind of change: graph trust (Phase 3) added a capability that simply was not there before — the ability to see coordination — while ML augmentation (Phase 5) was a refinement that cleared the last of the accuracy gap.

The robustness comes from how the signals combine. Because the user weight is a product of independent factors, an attacker has to beat all of them at the same time: stay clean on reliability tracking, avoid looking coordinated in the graph, keep a plausible location trail, behave normally on the temporal signals, and not trip the learned detector. Slipping past one is not enough, because the others still have a vote.

The conservatism is a deliberate trade, not an accident. Between the $T_i \leq R^*_i$ constraint and the five-condition gate, the system leans toward holding back. In practice that means a genuine report may occasionally take a little longer to reach a wide radius — but in the runs reported here, no false report reached one. For a tool whose job is to keep a community from being misled, that is the right side to err on.

The honest caveat is that all of this is measured in simulation. Real users will not behave exactly like the synthetic population, and the gap could cut either way. The system also makes no attempt to read the content of a report — no text or image forensics — and leans entirely on network-level signals; adding content analysis is the obvious way to help the cases where the social signal alone is genuinely ambiguous.

CHAPTER 5

CONCLUSION AND FUTURE WORKS

5.1 SUMMARY OF CONTRIBUTIONS

This report has described NCPS, a platform that swaps engagement for credibility as the thing that decides how local news spreads. Looking back over the work, five contributions stand out.

The first is the distribution model itself. Where a conventional feed amplifies whatever is popular, NCPS only widens a report's reach once it clears thresholds on credibility, evidence, voter agreement and geographic trust together. The organising question changes from "what is getting attention" to "what is probably true," and everything else in the design follows from that choice.

The second is the way user trust is assembled. A user's weight is the product of five independent factors — confidence-corrected Bayesian reliability, log-scaled experience, a five-signal anomaly score, graph-propagated trust with coordination detection, and spatial trust from location confidence. Because they multiply rather than add, a problem on any one of them is enough to pull a malicious account's influence toward zero; an attacker has to look clean on all five at once.

The third is the propagation gate. Five conditions must hold simultaneously before a report expands, and the whole thing is tuned so that wrongly spreading a falsehood is treated as worse than briefly holding back something true. That deliberate asymmetry is, as far as I have found, not something existing community-verification systems make explicit.

The fourth is methodological. Building the system in six phases and measuring after each one let me attribute the gains to specific components rather than to the system as a whole — a development pattern that should transfer to other trust-based systems beyond this one.

The fifth is simply that it runs. NCPS is a working application — authentication, the React interface, the interactive map, the REST API and the simulation harness — not a paper design or a notebook experiment.

67 — *Department of Computer Science and Engineering, National Institute of Technology Srinagar*

5.2 KEY FINDINGS

A few results from the evaluation are worth drawing out.

To start with, the Bayesian baseline is already good. Phase 1, with nothing but confidence-corrected credibility, reaches 0.900 accuracy — enough to confirm that the "users as noisy sensors" model with honest uncertainty handling is a sound footing. What it cannot do is hold up against organised attack: its 0.150 attack-success rate is the gap that the later phases exist to close.

The component that closes most of that gap is graph trust. Adding propagation and coordination detection in Phase 3 was the single most consequential change, because it is the first point at which the system can read structure — clusters of accounts moving in lockstep — instead of judging each voter in isolation. Everything before it is blind to collusion by construction.

The phases also show why no one signal is enough. Anomaly recall climbs $0.000 \rightarrow 0.480 \rightarrow 0.680 \rightarrow 0.800 \rightarrow 0.840$ as graph, spatial and behavioural signals come online, and the reason it keeps climbing is that each family catches a different slice of bad actors. The coverage comes from the combination, not from any single detector.

The learned layer earns its place at the end. The move from 0.920 to 1.000 in Phase 5 comes from the model picking up feature-space patterns that pure aggregation leaves ambiguous — it behaves like a corrective pass over the borderline cases rather than a replacement for the crowd estimate.

The behaviour under stress was the most reassuring part. Accuracy held at 1.000 even with 64% adversarial users, with bots at half the population, and with as few as 60 interactions. The confidence correction is doing quiet work here: it stops a flood of fresh, low-evidence malicious accounts from swinging a score, because thin evidence simply does not count for much.

Underneath all of it is one small rule with outsized effect: $T_i \leq R*_i$. Trust can fall through the graph but never rise above what a user has earned alone, so befriending trusted accounts buys an attacker nothing. That one-directional flow is the backbone of the system's resistance.

5.3 FUTURE WORK

The simulation results are encouraging, but they are still simulation results. Several lines of work would move NCPS closer to something that could be deployed and trusted in the field.

1. Real-World Dataset Validation: All of the numbers here come from synthetic data. The most important next step is to test against real traces — from Twitter, Reddit, or WhatsApp groups — where the hard parts are getting trustworthy ground-truth labels for local events and reproducing the messy distribution of real user behaviour rather than a tidy simulated one.

2. Content Analysis Integration: The current system does not analyze the textual or visual content of posts. Integrating NLP-based credibility signals (sentiment analysis, claim verification, source extraction) and image forensics (manipulated image detection) would add a powerful complementary dimension to the network-based signals.

3. Administrative Moderation Interface: A dedicated admin panel for human moderators to review flagged content, adjust credibility scores, and manage user accounts would be essential for real-world deployment. The admin interface should provide tools for escalation, appeal handling, and policy enforcement.

4. Real-Time Alerts and Notifications (Implemented): The current system includes SSE-based real-time alert streaming, an alerts inbox with unread/read state management, and Web Push subscription storage for registered users. Users receive immediate notifications for high-urgency, high-credibility posts in their vicinity. Future enhancements could extend this to native mobile push notifications and configurable email digest notifications for users who prefer periodic summaries.

5. Advanced Graph Neural Networks: Replacing the current EigenTrust-inspired propagation with more sophisticated GNN architectures (GraphSAGE, GAT) could improve trust estimation, particularly in sparse graphs where limited edge information makes simple propagation less effective.

6. Privacy-Preserving Location: The current location system collects and stores user coordinates, which raises privacy concerns. Implementing differential privacy mechanisms or zero-knowledge proofs for proximity verification would allow spatial trust computation without exposing exact user locations.

7. Explainable AI Module (Implemented): The system now provides a dedicated `/api/post/{id}/explain` endpoint that returns a full decision trace for any post's credibility assessment. The explanation includes component-level decomposition of `C_Bayes`, evidence mass, variance, individual vote weights, propagation gate conditions, and alert gate status. Future enhancements could add natural language explanations for non-technical users and interactive visual decision trees that allow users to explore the reasoning process step by step.

8. Mobile Application: Developing native mobile apps (iOS and Android) would dramatically increase the system's reach and usability for local news reporting, where users are typically on mobile devices at event locations.

9. Multi-Language Support: Extending the system to support multiple languages, particularly Indian languages (Hindi, Urdu, Kashmiri), would be crucial for deployment in the NIT Srinagar region and across India more broadly.

10. Continuous Threshold Learning: Currently, the system uses fixed thresholds (e.g., anomaly threshold = 0.3, propagation credibility threshold = 0.6). Implementing adaptive threshold learning that adjusts these values based on observed performance could improve the system's adaptability to different community dynamics.

11. Observability and Monitoring (Implemented): The system now includes structured request logging with correlation IDs, health and readiness check endpoints (`/api/health`, `/api/ready`), a metrics endpoint exposing key system statistics, and an admin observability panel for monitoring credibility distributions and system status. Future enhancements could integrate with industry-standard monitoring stacks such as Prometheus and Grafana for production-grade dashboards, distributed tracing, and automated alerting on anomalous system behavior.

12. Federation: Enabling multiple NCPS instances across different geographic regions to share trust information (while maintaining local autonomy) could create a federated verification network that combines the benefits of local community knowledge with broader credibility signals.

REFERENCES

- [1] S. Vosoughi, D. Roy, and S. Aral, "The spread of true and false news online," *Science*, vol. 359, no. 6380, pp. 1146-1151, 2018.
- [2] K. Shu, A. Sliva, S. Wang, J. Tang, and H. Liu, "Fake news detection on social media: A data mining perspective," *ACM SIGKDD Explorations Newsletter*, vol. 19, no. 1, pp. 22-36, 2017.
- [3] S. Banaji and R. Bhat, "WhatsApp Vigilantes: An exploration of citizen reception and circulation of WhatsApp misinformation linked to mob violence in India," *London School of Economics and Political Science*, 2019.
- [4] E. Grieco, "U.S. newsroom employment has dropped by a quarter since 2008, with greatest decline at newspapers," *Pew Research Center*, 2020.
- [5] N. Ruchansky, S. Seo, and Y. Liu, "CSI: A hybrid deep model for fake news detection," *Proceedings of the 2017 ACM Conference on Information and Knowledge Management (CIKM)*, pp. 797-806, 2017.
- [6] F. Monti, F. Frasca, D. Eynard, D. Mannion, and M. M. Bronstein, "Fake news detection on social media using geometric deep learning," *arXiv preprint arXiv:1902.06673*, 2019.
- [7] Y. Wang, F. Ma, Z. Jin, Y. Yuan, G. Xun, K. Jha, L. Su, and J. Gao, "EANN: Event adversarial neural networks for multi-modal fake news detection," *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 849-857, 2018.
- [8] Z. Jin, J. Cao, H. Guo, Y. Zhang, and J. Luo, "Multimodal fusion with recurrent neural networks for rumor detection on microblogs," *Proceedings of the 25th ACM International Conference on Multimedia*, pp. 795-816, 2017.
- [9] V. Perez-Rosas, B. Kleinberg, A. Lefevre, and R. Mihalcea, "Automatic detection of fake news," *Proceedings of the 27th International Conference on Computational Linguistics (COLING)*, pp. 3391-3401, 2018.
- [10] A. Gupta, P. Kumaraguru, C. Castillo, and P. Meier, "TweetCred: Real-time credibility assessment of content on Twitter," *Proceedings of the International Conference on Social Informatics (SocInfo)*, pp. 228-243, 2014.
- [11] O. Varol, E. Ferrara, C. A. Davis, F. Menczer, and A. Flammini, "Online human-bot interactions: Detection, estimation, and characterization," *Proceedings of the International AAAI Conference on Web and Social Media (ICWSM)*, vol. 11, no. 1, pp. 280-289, 2017.
- [12] A. Josang, R. Ismail, and C. Boyd, "A survey of trust and reputation systems for online service provision," *Decision Support Systems*, vol. 43, no. 2, pp. 618-644, 2007.
- [13] J. Ma, W. Gao, and K. F. Wong, "Rumor detection on Twitter with tree-structured recursive neural networks," *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 1980-1989, 2018.
- [14] Y. Li, J. Gao, C. Meng, Q. Li, L. Su, B. Zhao, W. Fan, and J. Han, "A survey on truth discovery," *ACM SIGKDD Explorations Newsletter*, vol. 17, no. 2, pp. 1-16, 2016.
- [15] D. Kempe, J. Kleinberg, and E. Tardos, "Maximizing the spread of influence through a social network," *Proceedings of the 9th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 137-146, 2003.

71 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

- [16] X. Yin, J. Han, and P. S. Yu, "Truth discovery with multiple conflicting information providers on the Web," *IEEE Transactions on Knowledge and Data Engineering*, vol. 20, no. 6, pp. 796-808, 2008.
- [17] J. Allen, A. A. Arechar, G. Pennycook, and D. G. Rand, "Scaling up fact-checking using the wisdom of crowds," *Science Advances*, vol. 7, no. 36, eabf4393, 2021.
- [18] Twitter, "Introducing Birdwatch, a community-based approach to misinformation," *Twitter Blog*, 2021. Available: https://blog.twitter.com/en_us/topics/product/2021/introducing-birdwatch-a-community-based-approach-to-misinformation

-
- [19] E. Cho, S. A. Myers, and J. Leskovec, "Friendship and mobility: User movement in location-based social networks," *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1082-1090, 2011.
- [20] M. F. Goodchild and J. A. Glennon, "Crowdsourcing geographic information for disaster response: A research frontier," *International Journal of Digital Earth*, vol. 3, no. 3, pp. 231-241, 2010.
- [21] S. D. Kamvar, M. T. Schlosser, and H. Garcia-Molina, "The Eigentrust algorithm for reputation management in P2P networks," *Proceedings of the 12th International Conference on World Wide Web (WWW)*, pp. 640-651, 2003.
- [22] Z. Gyöngyi, H. Garcia-Molina, and J. Pedersen, "Combating web spam with TrustRank," *Proceedings of the 30th International Conference on Very Large Data Bases (VLDB)*, pp. 576-587, 2004.
- [23] S. Cresci, R. Di Pietro, M. Petrocchi, A. Spognardi, and M. Tesconi, "The paradigm-shift of social spambots: Evidence, theories, and tools for the arms race," *Proceedings of the 26th International Conference on World Wide Web Companion (WWW)*, pp. 963-972, 2017.
- [24] L. Nizzoli, S. Tardelli, M. Avvenuti, S. Cresci, and M. Tesconi, "Coordinated behavior on social media in 2019 UK general election," *Proceedings of the International AAAI Conference on Web and Social Media (ICWSM)*, vol. 15, pp. 443-454, 2021.
- [25] T. Bian, X. Xiao, T. Xu, P. Zhao, W. Huang, Y. Rong, and J. Huang, "Rumor detection on social media with bi-directional graph convolutional networks," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 1, pp. 549-556, 2020.
- [26] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank citation ranking: Bringing order to the web," *Stanford InfoLab Technical Report*, 1999.
- [27] A. Ramachandran, D. Zocchi, and N. Feamster, "Filtering spam with behavioral blacklisting," *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS)*, pp. 342-351, 2007.
- [28] S. Tirunillai and G. J. Tellis, "Mining marketing meaning from online chatter: Strategic brand analysis of big data using latent Dirichlet allocation," *Journal of Marketing Research*, vol. 51, no. 4, pp. 463-479, 2014.
- [29] S. Ramalingam et al., "Tiagra: A toolbox for integrated analysis of geocoded research data," *SoftwareX*, vol. 12, p. 100588, 2020.
- [30] FastAPI Documentation, "FastAPI - Modern, Fast (high-performance), Web Framework," Available: <https://fastapi.tiangolo.com>, 2024.
- [31] React Documentation, "React - A JavaScript library for building user interfaces," Available: <https://react.dev>, 2024.
- [32] Leaflet.js Documentation, "Leaflet - an open-source JavaScript library for mobile-friendly interactive maps," Available: <https://leafletjs.com>, 2024.
-

APPENDIX A

COMPLETE API ENDPOINT REFERENCE

The following table lists all REST API endpoints exposed by the NCPS backend, organized by functional category:

S.No	Route	Method	Auth Required	Description
1	/api/auth/register	POST	No	Register a new user account with email and password
2	/api/auth/login	POST	No	Authenticate and receive JWT token
3	/api/auth/me	GET	Yes	Get current authenticated user details
4	/api/users/	GET	No	List all users with basic profile information
5	/api/users/{user_id}	GET	No	Get detailed user profile including trust signals
6	/api/users/{user_id}/state	GET	No	Get user's current computed state (R, R*, Conf, Exp, etc.)
7	/api/posts/	POST	Yes	Create a new news report with content and location
8	/api/posts/	GET	No	List all posts with computed credibility scores
9	/api/posts/{post_id}	GET	No	Get detailed post information including voter breakdown
10	/api/posts/{post_id}/vote	POST	Yes	Cast a vote (+1 or -1) on a post
11	/api/feed	GET	No	Get credibility-ranked feed of posts
12	/api/map/data	GET	No	Get geographic data for map visualization (posts, users, radii)
13	/api/profile/{user_id}	GET	Yes	Get authenticated user's full profile with trust decomposition
14	/api/analytics/overview	GET	No	Get system-wide analytics (credibility distribution, user stats)
15	/api/simulation/run	POST	No	Trigger a simulation run with specified parameters
16	/api/simulation/results	GET	No	Get results of the last simulation run
17	/api/health	GET	No	Health check endpoint returning system status

Authentication: Endpoints marked with "Auth Required = Yes" expect a valid JWT token in the Authorization header with the format: `Authorization: Bearer <token>`. Tokens are obtained through the `/api/auth/login` endpoint and have a configurable expiration period (default: 24 hours).

Error Handling: All endpoints return standard HTTP status codes: - 200: Success - 201: Created (for POST operations) - 400: Bad Request (invalid input, duplicate vote, etc.) - 401: Unauthorized (missing or invalid token) - 404: Not Found (invalid user_id or post_id) - 500: Internal Server Error

APPENDIX B

HYPERPARAMETER REFERENCE

The following table lists all configurable hyperparameters in the NCPS system, their default values, and descriptions:

Table B.1: Complete Hyperparameter Reference

Parameter Name	Default Value	Description
User Engine		
α_0 (prior_alpha)	1.0	Initial positive Bayesian evidence count
β_0 (prior_beta)	1.0	Initial negative Bayesian evidence count
k (evidence_growth_rate)	0.1	Rate parameter for confidence saturation
E_max (max_experience_count)	100	Maximum experience normalization value
Post Engine		
α_0 (post_prior_alpha)	1.0	Post credibility prior (positive)
β_0 (post_prior_beta)	1.0	Post credibility prior (negative)
λ_{decay} (temporal_decay_rate)	0.01	Exponential decay rate for vote aging
Graph Engine		
λ (damping_factor)	0.5	Trust propagation damping factor
max_iterations	50	Maximum trust propagation iterations
tolerance (ϵ)	1e-6	Convergence tolerance for trust propagation
coord_threshold	0.7	Coordination detection threshold
link_dampen_factor	0.1	Edge weight reduction for coordinated links
Spatial Engine		
σ (proximity_sigma_km)	5.0	Gaussian proximity standard deviation (km)
max_devices	5	Maximum normal device count
max_ips	10	Maximum normal IP address count
base_radius_km	2.0	Initial post propagation radius
max_radius_km	50.0	Maximum allowed propagation radius
radius_step_km	0.5	Radius expansion step per evaluation
ML Engine		
α_{ml} (ml_weight)	0.15	ML prediction weight in final credibility
γ_{mem} (memory_weight)	0.10	Memory score weight in final credibility
min_training_samples	10	Minimum resolved posts before ML training
Decision Engine		
credibility_threshold	0.6	Minimum credibility for propagation
evidence_threshold	3.0	Minimum evidence mass for propagation
variance_threshold	0.25	Maximum variance for propagation
min_age_seconds	60	Minimum post age for propagation
location_trust_threshold	0.3	Minimum mean voter location trust
Anomaly Detection		

Parameter Name	Default Value	Description
anomaly_threshold	0.3	Score above which user is considered anomalous
burst_window_seconds	60	Time window for burst detection
max_actions_per_window	10	Maximum normal actions per burst window
Simulation		
num_honest	40	Default honest user count
num_noisy	5	Default noisy user count
num_adversarial	5	Default adversarial user count
num_bots	20	Default bot count
num_posts	50	Default post count
num_interactions	1000	Default interaction count

78 — Department of Computer Science and Engineering, National Institute of Technology Srinagar

APPENDIX C

SETUP AND EXECUTION GUIDE

This appendix provides instructions for setting up and running the NCPS platform on a local development environment.

C.1 PREREQUISITES

Ensure the following software is installed: - **Python** 3.10 or higher - **Node.js** 18+ and npm 9+ - **PostgreSQL** 14+ - **Git** for version control

C.2 BACKEND SETUP

```
# 1. Clone the repository
git clone https://github.com/Niteshkrjhag/NCPS.git
cd NCPS

# 2. Create a Python virtual environment
python3 -m venv venv
source venv/bin/activate    # Linux/macOS
# or: venv\Scripts\activate # Windows

# 3. Install backend dependencies
pip install -r requirements.txt

# 4. Configure environment variables
cp .env.example .env
# Edit .env with your PostgreSQL credentials:
# DATABASE_URL=postgresql://user:password@localhost:5432/ncps_db
# SECRET_KEY=your-secret-key-here
# ALGORITHM=HS256
# ACCESS_TOKEN_EXPIRE_MINUTES=1440
```

C.3 DATABASE SETUP

```
# 1. Create the PostgreSQL database
createdb ncps_db

# 2. Initialize database schema
# The application auto-creates tables on first run via SQLAlchemy
python -c "from backend.app.database import engine, Base;
Base.metadata.create_all(bind=engine)"
```

C.4 BACKEND EXECUTION

```
# Start the FastAPI server
cd backend
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

# The API is now accessible at http://localhost:8000
# API documentation at http://localhost:8000/docs (Swagger UI)
# Alternative docs at http://localhost:8000/redoc
```

C.5 FRONTEND SETUP (VANILLA JAVASCRIPT)

```
# The vanilla JS frontend is served as static files
# Open directly in browser or use a simple HTTP server
cd frontend
python3 -m http.server 5500

# Access at http://localhost:5500
```

C.6 REACT FRONTEND SETUP

```
# 1. Install React dependencies
cd react-frontend
npm install

# 2. Start development server
npm run dev

# Access at http://localhost:5173
```

C.7 RUNNING THE SIMULATION

```
# Option 1: Run via API endpoint
curl -X POST http://localhost:8000/api/simulation/run

# Option 2: Run directly via Python
python -m backend.app.simulation.run_simulation

# Option 3: Use the web-based simulation dashboard
# Navigate to /simulation in the webapp
```

C.8 RUNNING TESTS

```
# Run the full test suite
cd backend
python -m pytest tests/ -v

# Run specific test categories
python -m pytest tests/test_user_engine.py -v
python -m pytest tests/test_post_engine.py -v
python -m pytest tests/test_graph_engine.py -v
python -m pytest tests/test_simulation.py -v
```

C.9 PROJECT DIRECTORY STRUCTURE

```

NCPS/
├── backend/
│   ├── app/
│   │   ├── main.py           # FastAPI application entry point
│   │   ├── database.py       # Database connection and session management
│   │   ├── models.py         # SQLAlchemy ORM models
│   │   ├── auth.py           # JWT authentication logic
│   │   └── engine/
│   │       ├── user_engine.py # User weight computation
│   │       ├── post_engine.py # Post credibility computation
│   │       ├── graph_engine.py # Graph trust and coordination detection
│   │       ├── spatial.py     # Spatial trust and propagation
│   │       ├── ml_engine.py   # ML and memory augmentation
│   │       ├── signal_engine.py # Behavioral signal computation
│   │       ├── urgency.py     # Urgency classification
│   │       └── decision.py    # Propagation decision engine
│   ├── routes/
│   │   ├── auth.py           # Authentication endpoints
│   │   ├── users.py          # User management endpoints
│   │   ├── posts.py          # Post and voting endpoints
│   │   ├── feed.py           # Feed retrieval endpoints
│   │   ├── map.py            # Map data endpoints
│   │   └── analytics.py      # Analytics endpoints
│   ├── simulation/
│   │   └── run_simulation.py  # Simulation framework
│   └── webapp/
│       ├── static/           # Static files (CSS, JS, images)
│       └── templates/        # HTML templates
├── tests/                    # Test suite
├── requirements.txt          # Python dependencies
├── frontend/                 # Vanilla JS frontend
│   ├── index.html
│   ├── css/
│   └── js/
├── react-frontend/           # React frontend
│   ├── src/
│   ├── package.json
│   └── vite.config.js
├── docs/                     # Documentation
│   └── context/               # Project context files
├── CHANGELOG.md
├── hyperparameter.md
└── README.md

```